ETH zürich



Marco Ferroni's Portfolio

Profile

I’m a 24 years old passionate student from a small town near Ravenna (IT).



- **Education**
  - **Master Degree:** Electrical Engineering at ETH Zurich (Final GPA 5.7/6).
  - **Bachelor Degree:** Electronics Engineering at Politecnico di Milano (110L/110 with 29.48/30 final GPA).
- **Scholarships**
  - **Best freshmen award:** 1000 euros scholarship awarded to the best freshmen of Politecnico di Milano (top 2.5%).
  - **Fee exemption:** in my Bachelor due to my high GPA (>29/30) I was granted a fee exemption.
- **Job experience**
  - **Visiting Researcher at Stanford University:** I spent six months as part of the Brains in Silicon Lab, where I worked on my master's thesis.
  - **Circuit Design Intern at SynSense:** I was employed for six months as part of the IC design team, where I’ve designed full-custom logic blocks.

ETH zürich



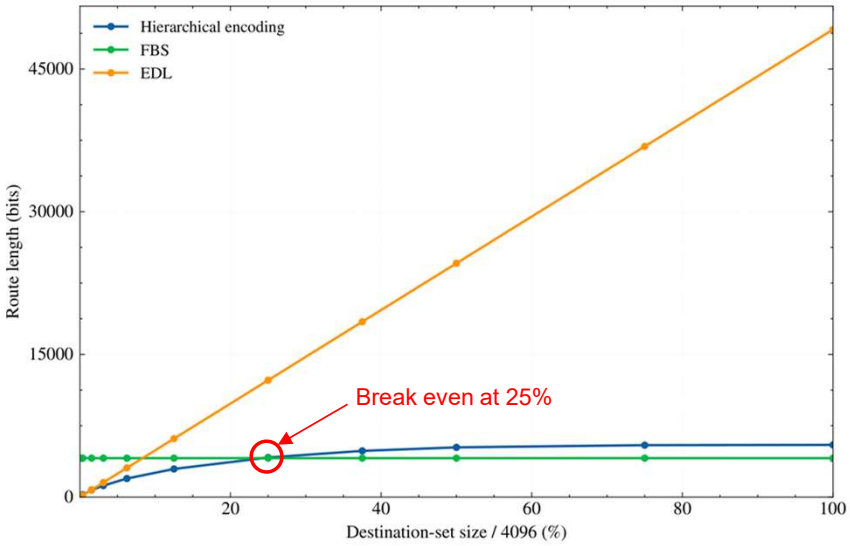
An Asynchronous Tree Network-on-Chip for Exact Hierarchical Multicast

Design of a quad-tree NoC -Overview

- **Context**
  - **Master's thesis project:** pursued at Stanford University under the supervision of Prof. Kwabena Boahen.
  - **Scope:** Implement a quad-tree asynchronous NoC with multicasting support, exploring a novel route encoding scheme
- **Goal**
  - Go from Python model to transistor level simulation
- **Workflow**
  - **Architecture exploration:** test ideas using a Python model
  - **CHP writing:** use CHP (Communicating Hardware Processes), a high-level description language designed for asynchronous circuits.
  - **Decomposition:** break the CHP design down into smaller, simpler blocks.
  - **PRS writing:** describe the design at the logic level using PRS, a lower-level abstraction.
  - **Layout and GDS:** finally as last step I will produce the layout and gds

Introduction

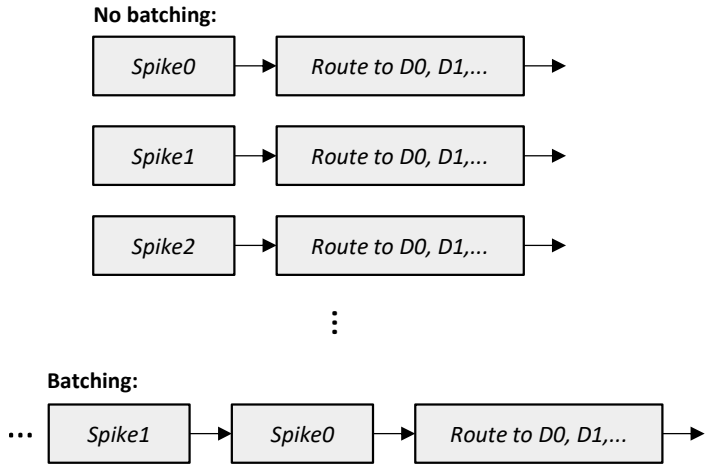
- **Networks-on-Chip** (NoCs) are ubiquitous today and
  - No one-size-fits-all solution
- Our target: **Spiking Neural Networks** (SNNs)
  - Globally sparse and locally dense communication
- Sparsity → asynchronous logic
- Communication pattern → tree topology
- Multicasting is a natural fit
  - Approximate multicasting:
    - Fewer routing bits → lower energy and latency
    - False paths → additional energy and congestion
    - False deliveries → additional energy for filtering
  - Exact multicasting:
    - More routing bits → higher energy and latency
    - No false paths → lower energy and congestion
    - No false deliveries → no local filtering
- The tradeoffs strongly depend on the length of the payload
- We focus on **exact multicasting**
- Current routing schemes scale poorly with the % of destinations
  - Our routing scheme, **hierarchical encoding**, addresses this limitation



Route length of different exact multicast routing schemes VS percentage of destinations

Introduction

- SNN spike packets typically have very short payloads
  - Often just a presynaptic neuron address
- Some SNN workloads naturally provide strong batching opportunities
  - Multiple spikes from the same source tile share the same destination set
  - Their source addresses can share a single multicast route
  - Multiple multicast transactions → **one batched transaction**
- Batching amortizes the cost of exact multicast routing
  - With sufficient batching, our exact scheme reduces dynamic NoC energy relative to **Hierarchical Bit String (HBS)**
- Contributions:
  - Circuit-level implementation of a fully asynchronous router with native unicast and multicast support
  - A novel hierarchical routing scheme for exact multicast
  - A multi-level evaluation of the proposed design



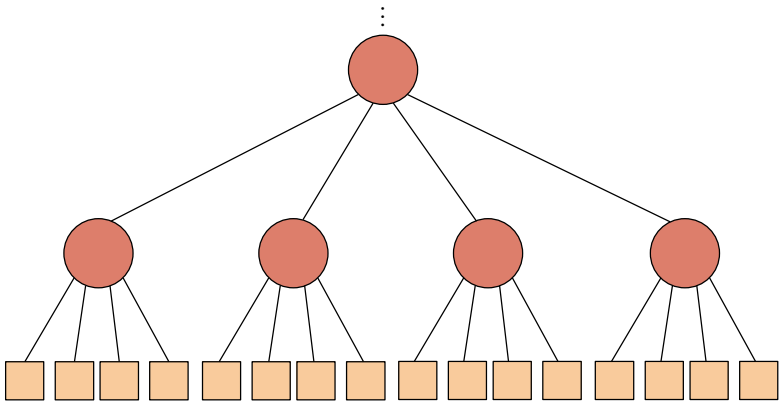
Distinction between batching and no batching. Each "spike" corresponds to the address of a presynaptic neuron

Introduction - Contributions

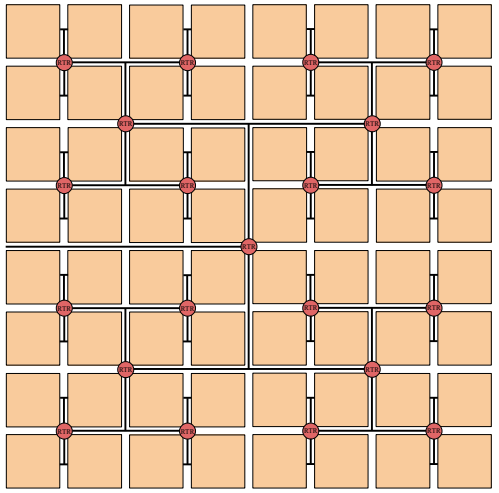
1. **Circuit-level implementation** of a fully asynchronous router with native unicast and multicast support
   a. The design was developed from the initial concept down to a transistor-level SPICE netlist
   b. The design flow progressively refined the architecture across multiple abstraction levels:
      i. Communicating Hardware Processes (CHP)
      ii. Handshaking Expansion (HSE)
      iii. Production Rules Set (PRS)
      iv. Transistor-level SPICE netlist
2. A **novel hierarchical routing scheme** for exact multicast
   a. Represents arbitrary destination sets without false-positive deliveries
   b. Routing overhead scales with the number and structure of the active destinations
3. A **multi-level evaluation** of the proposed design
   a. Circuit-level router characterization
      i. Forward latency L_f
      ii. Initiation interval/ cycle time II
      iii. Dynamic energy E
   b. Application-level benchmarks: evaluation using communication traces generated from SNN inference workloads

Network architecture - Topology

- The proposed NoC is logically organized as a quad-tree
  - Each internal router has four children
  - Endpoints are located at the leaves and can act as both sources and destinations
- The network is physically embedded as an H-tree
  - This mapping follows naturally from a regular 2D tile array
- Advantages:
  - **Structural scaling:** the number of router stages between endpoints grows logarithmically with network size with distance
  - **Deadlock-free routing:** packets travel **up** toward the LCA and then only **down**
- Disadvantages:
  - **Expensive upper-level interconnects:** links become progressively longer toward the root
  - **Congestion near the root:** global traffic is concentrated onto fewer high-level resources



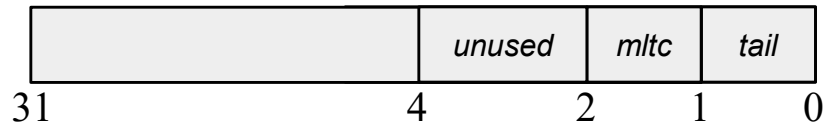
Logical implementation as a graph



Physical implementation as an H-tree

Network architecture - Communication fundamentals

- Each transferred word is a **flit**
- The network uses wormhole routing
  - An initial **header flit** establishes the route through the network
  - The remaining flits then follow the established path
- The bandwidth is fixed to at W=32 bits
- The four least-significant bits are reserved for control
  - **Bit 0 — tail:** asserted on the final flit of the transmission
  - **Bit 1 — mltc:** asserted when the flit carries multicast-routing information
  - **Bits 2–3:** currently unused and reserved for future extensions

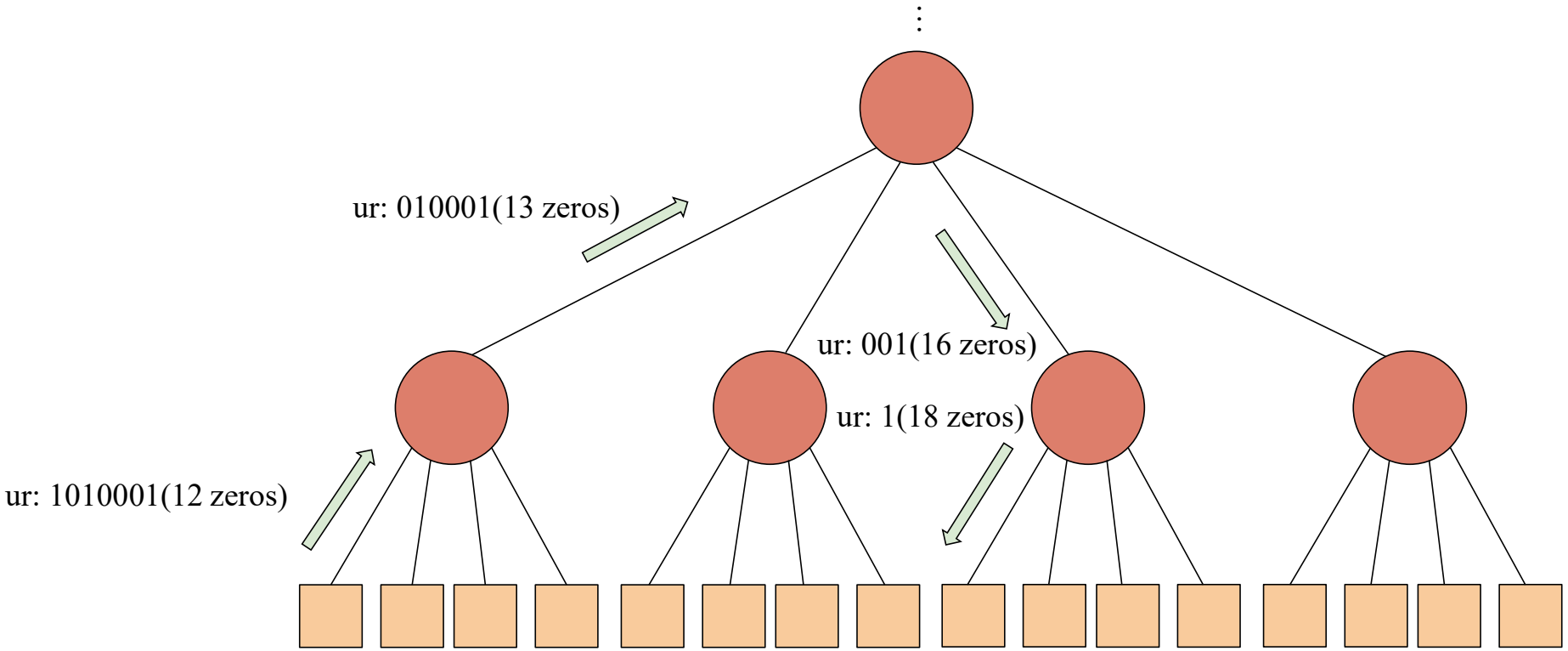


Control bits in a flit

Network architecture – Unicast communication

- Unicast routing follows an up–down path
  - **Up phase:** the packet climbs the tree until it reaches the **lowest common ancestor (LCA)**
  - **Down phase:** it then descends toward the destination, selecting one of four children at each level
- The 19 MSBs of the header encode the unicast route (ur)
  - Routing decisions are progressively **consumed by shifting** ur
  - Worst case: **5 upward steps + 1 direction-reversal + 6 × 2 child-selection bits + 1 termination bit = 19 bits**
- The final route bit acts as a termination marker
  - When it reaches the MSB it forms the **unicast stop code (USCODE)**
  - The detection of **USCODE** inside a router switches from unicast routing to **multicast processing**

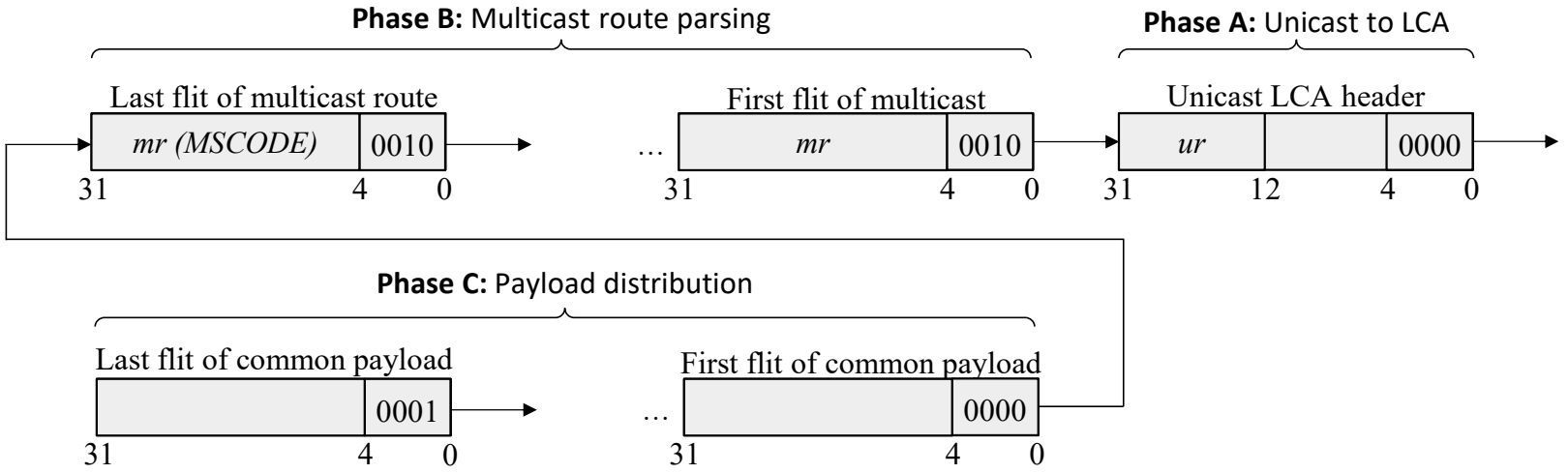
Network architecture – Unicast communication



Unicast routing example

Network architecture – Multicast communication

- Multicast routing proceeds in three phases
  a. **Unicast to the multicast root (LCA):** the packet starts with a unicast header that routes it to the lowest common ancestor of the destination set, where **USCODE** is detected
  b. **Multicast route parsing:** the router discards the header and parses the following multicast-route flits
  c. **Payload distribution:** once the multicast stop code (**MSCODE**) is detected, the remaining payload flits are replicated toward all active children



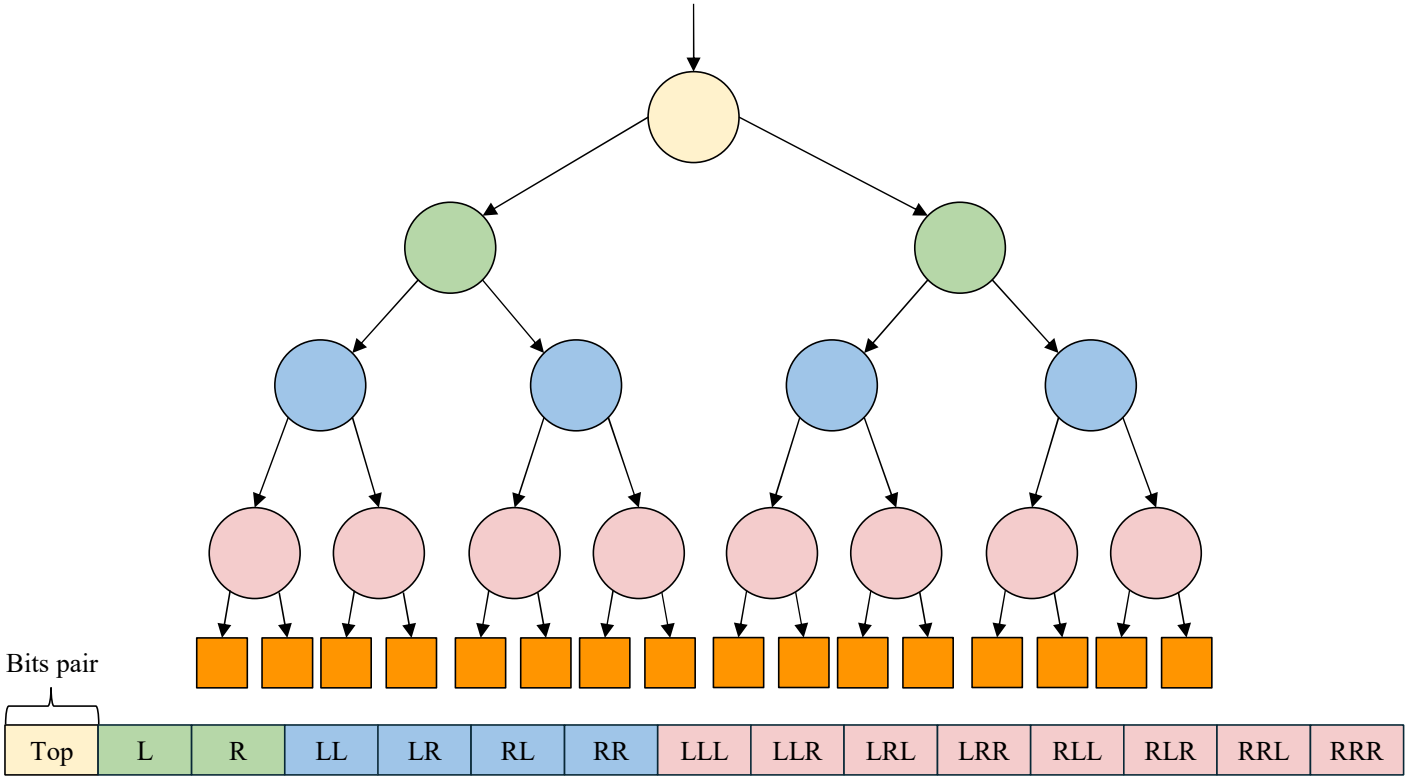
Structure of a multicast transmission

Hierarchical routing scheme - Assumptions

- The multicast route is represented as a continuous bit stream
  - Processed from left to right
- We consider the route from the perspective of a router high enough in the tree that all remaining communication proceeds downward.
- Each router is shown with two children instead of four
  - This simplifies the representation without changing the underlying principle
- Each router belonging to the multicast subtree is represented by a 2-bit child-selection mask
  - One bit corresponds to each child
  - **0:** the child is inactive
  - **1:** the message is forwarded to that child
  - If both bits are 0, that's the stopcode of the route (MSCODE)

Hierarchical routing scheme - Encoding

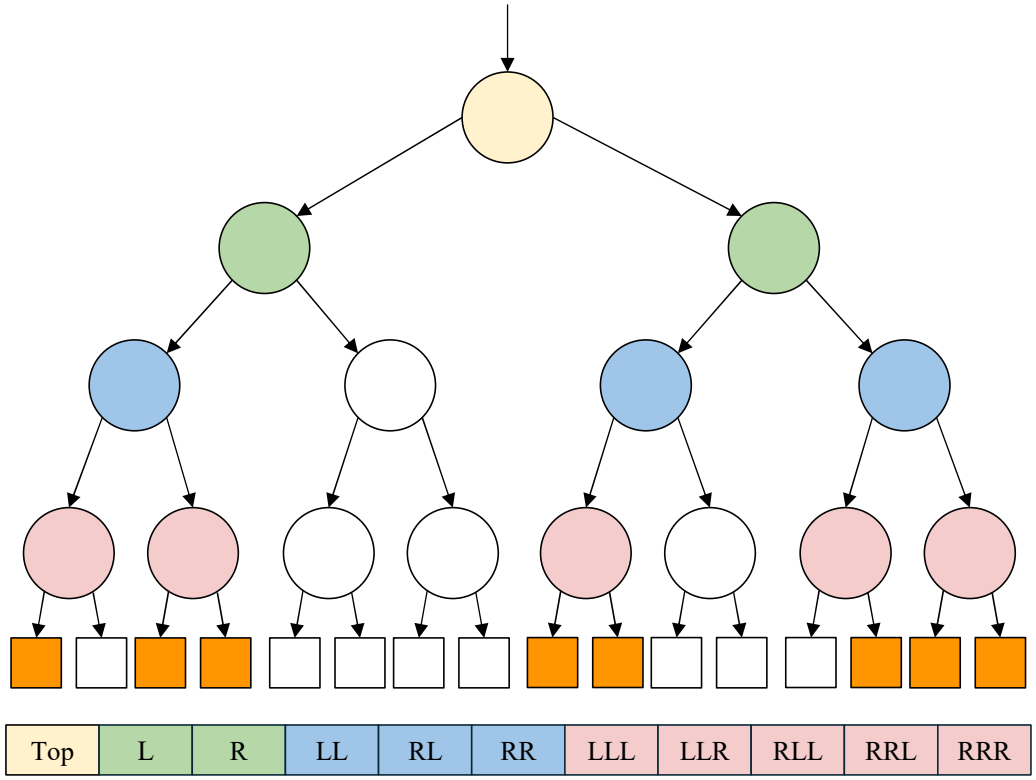
- **Broadcast case:**
  - Every destination in the subtree receives the common payload
  - Therefore, the child-selection mask of every internal node in the subtree is included in the multicast route



Route encoding for the broadcast case

Hierarchical routing scheme - Encoding

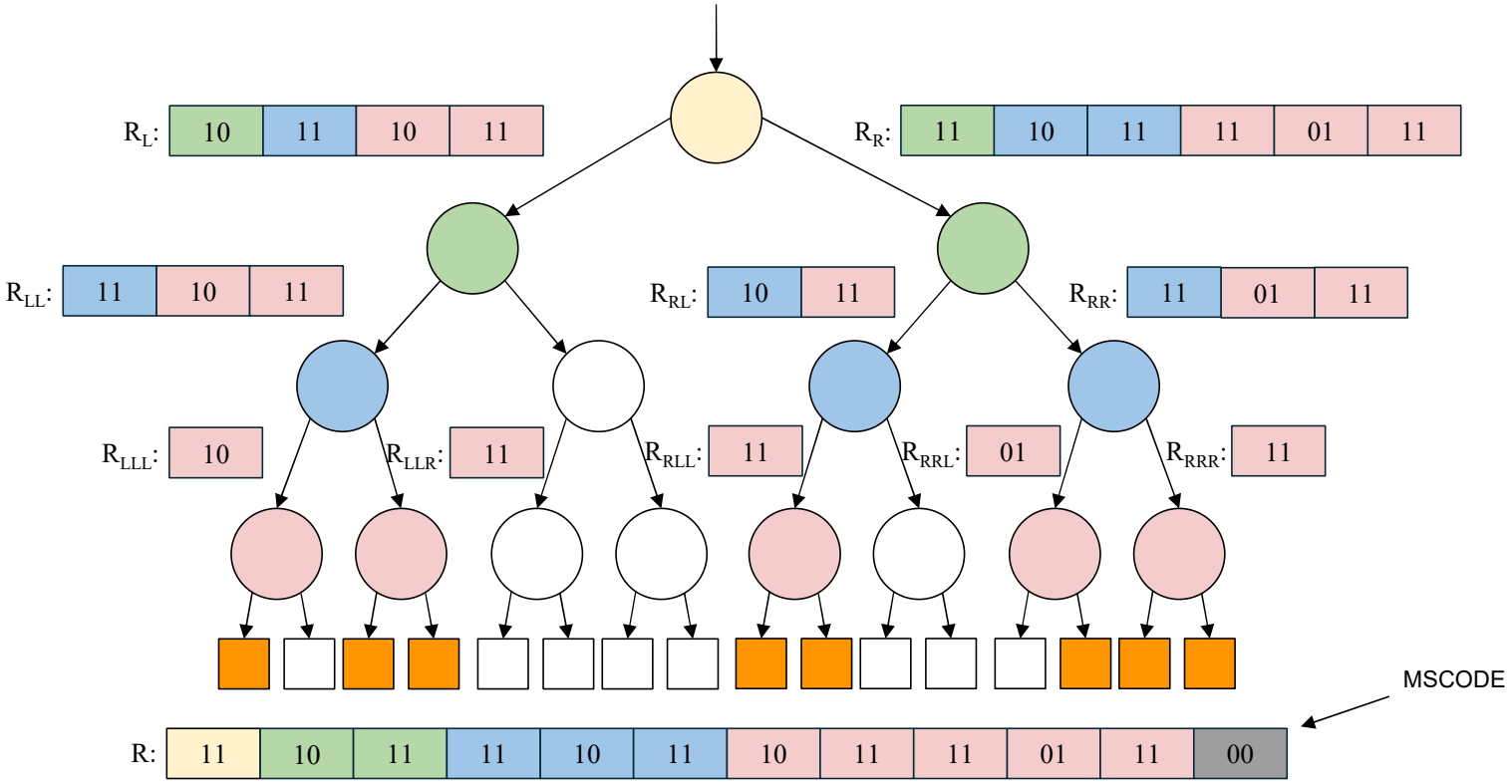
- **Multicast case:**
  - Some destinations in the subtree don’t receive the common payload
  - Therefore, come child-selection mask are skipped → **Compression**



Route encoding for the multicast case

Hierarchical routing scheme - Parsing

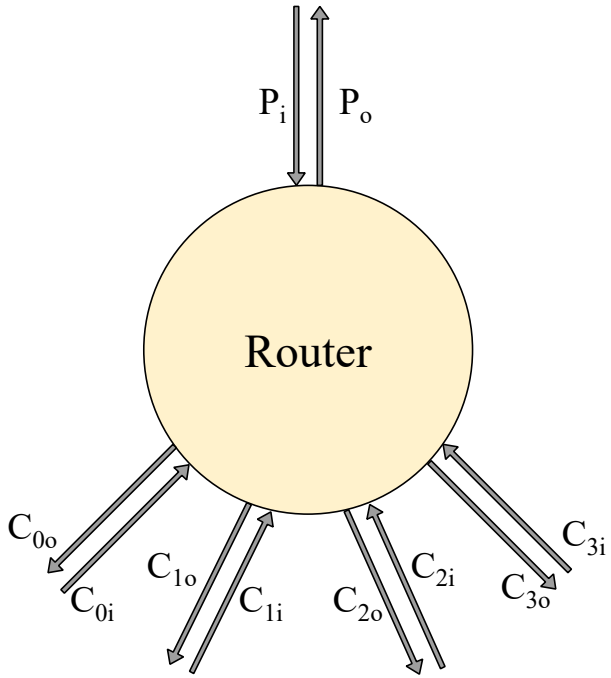
- Each router parses the route received from its parent into two child subroutes
  - The subroutes R_L and R_R are built level by level by scanning the masks in R from left to right
  - The number of masks belonging to R_i at level n, depends on the number of “1”s counted at level n-1



Route encoding for the multicast case

Router microarchitecture - Top level

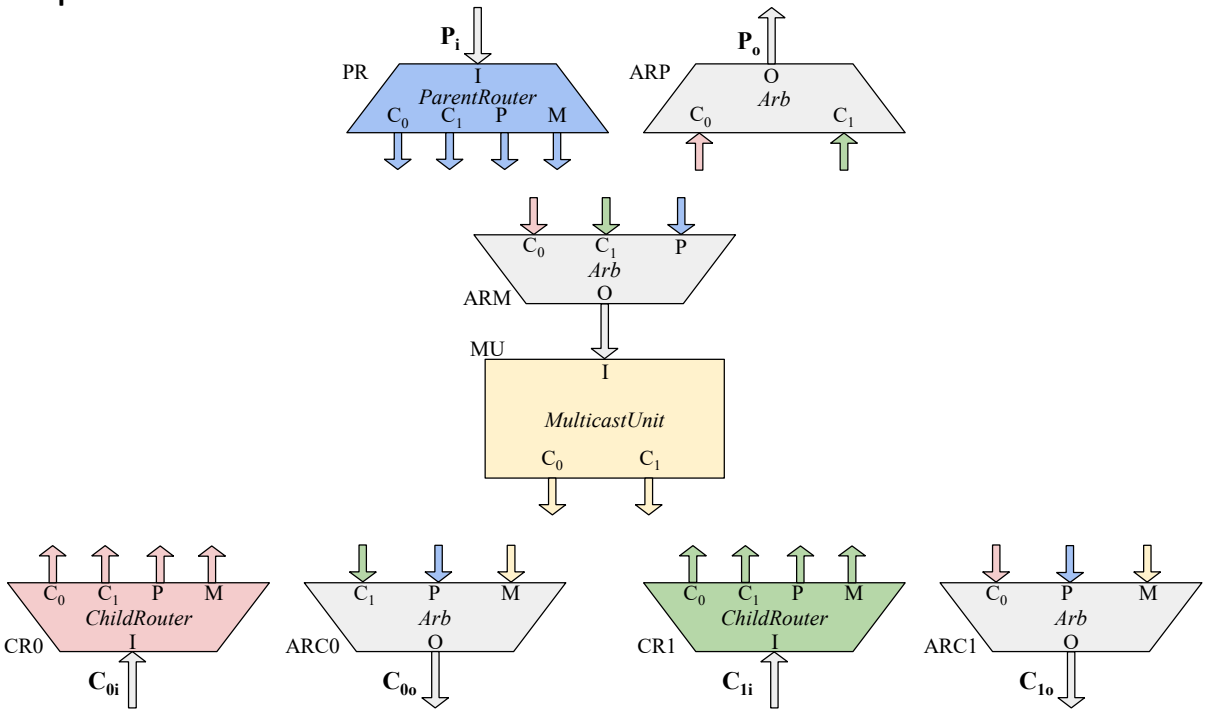
- Each router node has five full-duplex ports
  - 1 parent port + 4 child ports
  - each unidirectional channel has a **32-bit datapath**



Router's top view

Router microarchitecture - Top decompostion

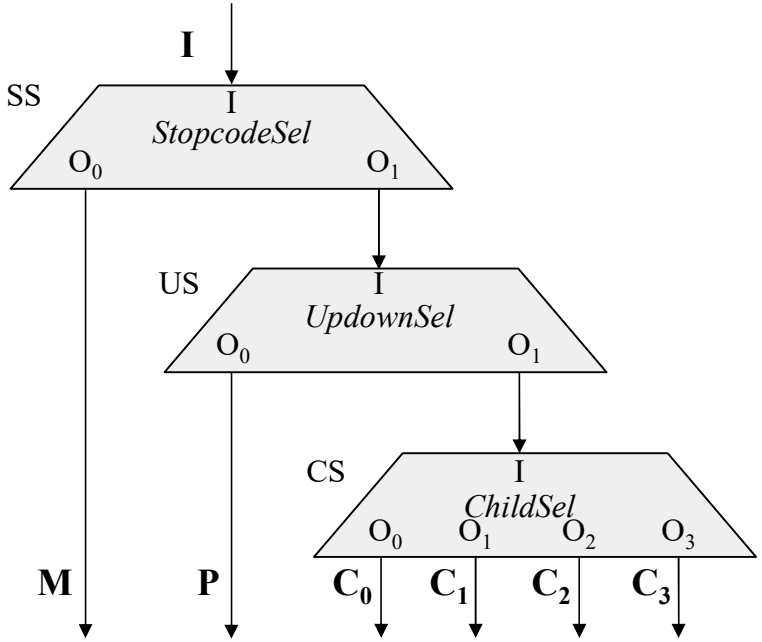
- Arbitration is required when multiple messages request the same output port or shared internal resource
- Messages arriving from the parent and child ports require different routing logic
  - Implemented through separate **ParentRouter** and **ChildRouter** blocks
- Multicast transmissions require a dedicated datapath compared to normal unicast
  - This additional datapath is called **MltcUnit**



Router's top decomposition

Router microarchitecture - ChildRouter

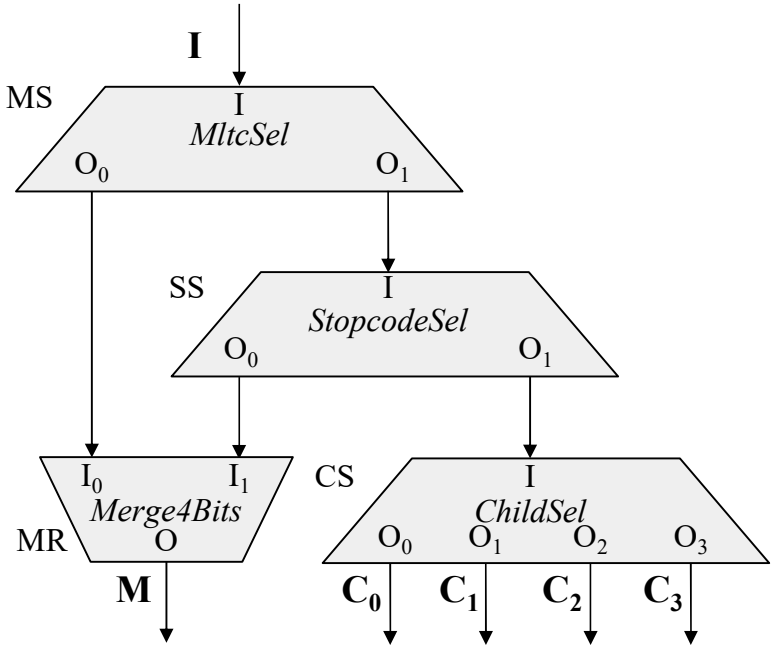
- A transmission arriving from a child port can follow three paths
  - **Multicast transition:** ur = USCODE → forward to the **MltcUnit**
  - **Upward unicast:** MSB = 1 → forward to the parent
  - **Downward unicast:** MSB = 0 → select one of the child ports
- The ChildRouter therefore behaves as a hierarchical controlled split
  - Its routing decision is decomposed into a cascade of simpler selector stages



ChildRouter decomposition

Router microarchitecture - ParentRouter

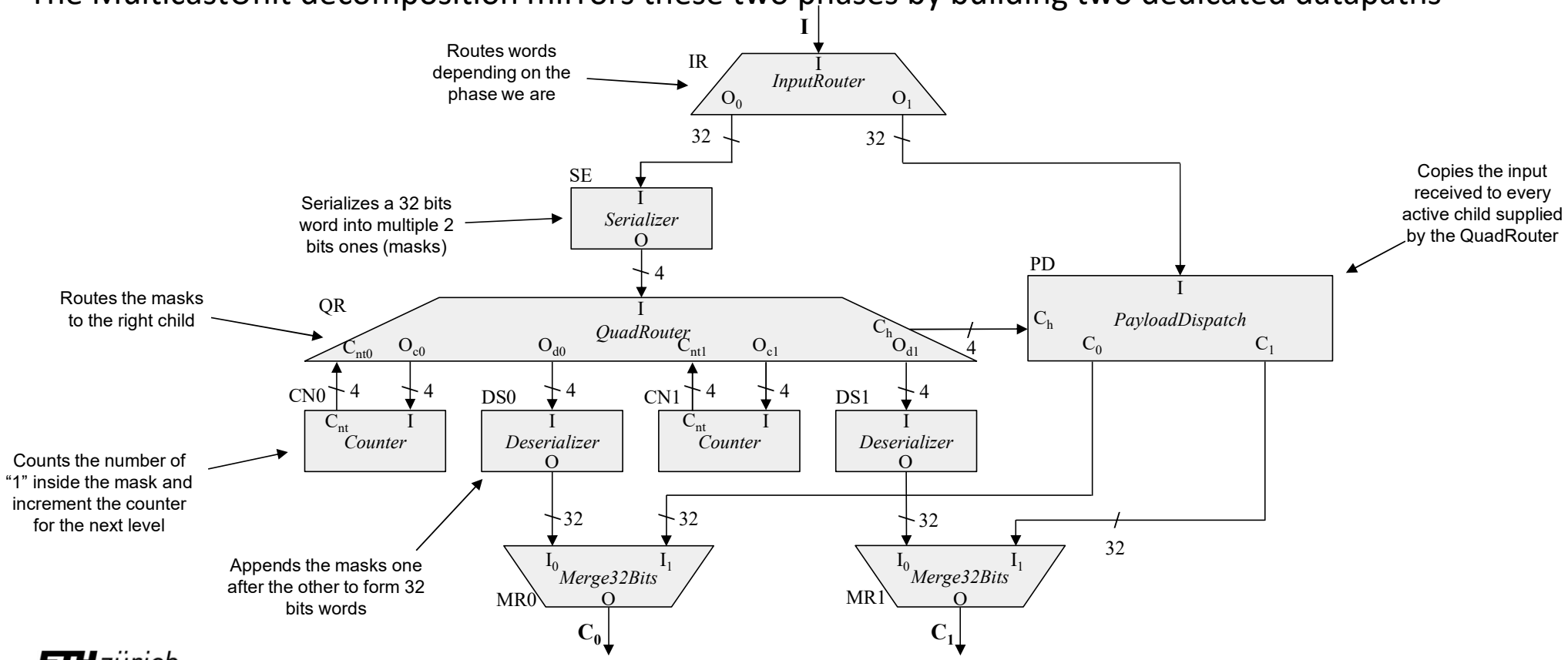
- A transmission arriving from the parent port can fall in three cases
  - **Multicast route parsing:** mltc = 1→ forward to the **MltcUnit** without skipping the first flit
  - **Multicast transition:** ur = USCODE → forward to the **MltcUnit**
  - **Downward unicast:** none of the above → select one of the child ports
- The ParentRouter behaves as a hierarchical controlled split as well
  - Its routing decision is decomposed into a cascade of simpler selector stages



ParentRouter decomposition

Router microarchitecture - MltcUnit

- An incoming multicast stream contains two consecutive phases
  - **Route setup:** parse and redistribute the multicast route toward the active children
  - **Payload distribution:** replicate the common payload toward those same children
- The MulticastUnit decomposition mirrors these two phases by building two dedicated datapaths



MltcUnit decomposition assuming 2 children

Implementation flow

1. CHP — Behavioral specification: concurrent processes communicate through **atomic handshake channels**
   a. Complete router specified and **functionally verified in ACTsim**
   b. Decomposed into smaller concurrent blocks for implementability
   c. Bit slicing omitted: 32-bit datapaths remain jointly completion-detected
2. HSE — Signal-level protocol: abstract communications → explicit **Boolean signal transitions**
   a. Communications implemented with **dual-rail, four-phase return-to-zero** handshaking
   b. Reshuffling to expose concurrency and simplify implementation
   c. State-variable insertion to distinguish otherwise ambiguous states
3. PRS — Circuit-level representation: boolean guards directly control circuit-node transitions
   a. Ensure **stability, non-interference, and correct reset behavior**
   b. PRS optimization and completion-tree decomposition
   c. Isochronic-fork analysis through randomized-delay stress testing in PRSIM
   d. Bubble reshuffling to obtain a CMOS-implementable PRS
   e. Functional verification of the final PRS
4. SPICE — Transistor-level implementation
   a. Final PRS → CMOS transistor netlist using prs2net
   b. PTM65 TT, V_{DD}=1.1V, 27°C
   c. HSPICE verification and characterization

Results - Methodology

- **Router characterization:** transistor-level HSPICE simulations
  - PTM 65 nm, TT corner, V_{DD}=1.1V, 27°C
  - Measured: forward latency $\mathbf{L_f}$, initiation time **II** and dynamic energy per bit **E** for different internal paths
- **Application benchmarks:** evaluation using realistic SNN inference traces with high batching opportunity
  - Discrete-event Python simulator parameterized by **SPICE-derived LUTs**
    - **16 tiles**, target capacity of **256 neurons/tile**
  - Physical link delay and energy modeled from distributed-RC wire parameters
  - Workloads: **MAP-SNN / N-MNIST**, **MAP-SNN / SHD**, **LSM / N-MNIST**
  - Addresses are batched within the same timestep
  - Batch size: $B_{max} \in \{1,2,4,8,16,32,64\}$
  - Compared against:
    - **22-nm asynchronous HBS tree NoC** using published post-layout timing/energy
    - **22-nm synchronous multicast-capable mesh (Xpipes)**, using published post-layout timing/energy
  - All architectures use the **same mapping and spike traces**; no technology normalization is applied

Results - Router characterization

Timing - Unicast

II up to 6× larger than forward latency due to poorly optimized recovery time

Path	Flit type	L_f [ns]	II [ns]
$C_i \rightarrow C_o$	Header	2.291–2.611	3.497–3.512
	Data	0.822–0.825	3.221–3.247
	Tail	0.834–0.838	3.151–3.164
$C_i \rightarrow P_o$	Header	1.737–1.845	3.491–3.500
	Data	0.530–0.533	3.104–3.113
	Tail	0.537–0.539	3.149–3.159
$P_i \rightarrow C_o$	Header	2.265–2.324	2.691–2.701
	Data	0.789–0.791	2.492–2.498
	Tail	0.795–0.799	2.364–2.374

Low forward latency except for the headers

Timing - Multicast Payload Dispatch

II up to 5× larger than forward latency

Input	Flit type	L_f [ns]	II [ns]
C_i	First data flit	1.378–1.480	4.891–4.950
	Intermediate data	0.875–0.880	4.186–4.191
	Tail	0.877–0.880	4.208–4.212
P_i	First data flit	1.390–1.391	1.604–1.606
	Intermediate data	0.886–0.889	3.211–3.213
	Tail	0.887–0.890	3.233–3.240

Low forward latency except for the headers

Timing - Route parsing

Parser phase	II [ns]
Initial route word	32.713–74.853
Continuation route word	23.115–51.273

Route parsing is expensive and highly data-dependent.

Energy per bit - Unicast

Path	Flit type	E [fJ/bit]
$C_i \rightarrow C_o$	Header	149.291–159.311
	Data	131.409–132.185
	Tail	138.211–138.966
$C_i \rightarrow P_o$	Header	92.088–93.308
	Data	79.967–80.393
	Tail	83.295–83.601
$P_i \rightarrow C_o$	Header	137.071–141.977
	Data	120.586–120.838
	Tail	126.373–126.807

Energy consumptions mostly dependent on input port

Energy per bit - Multicast Payload Dispatch (F is the fanout/ active children)

Input	Flit type	$F = 1$ [fJ/bit]	$F = 4$ [fJ/bit]
C_i	First data flit	178.237–179.688	359.932–364.211
	Intermediate data	173.998–174.177	347.575–347.876
	Tail	176.624–176.806	351.692–352.060
P_i	First data flit	183.831	365.401
	Intermediate data	179.269	353.135
	Tail	181.768	356.697

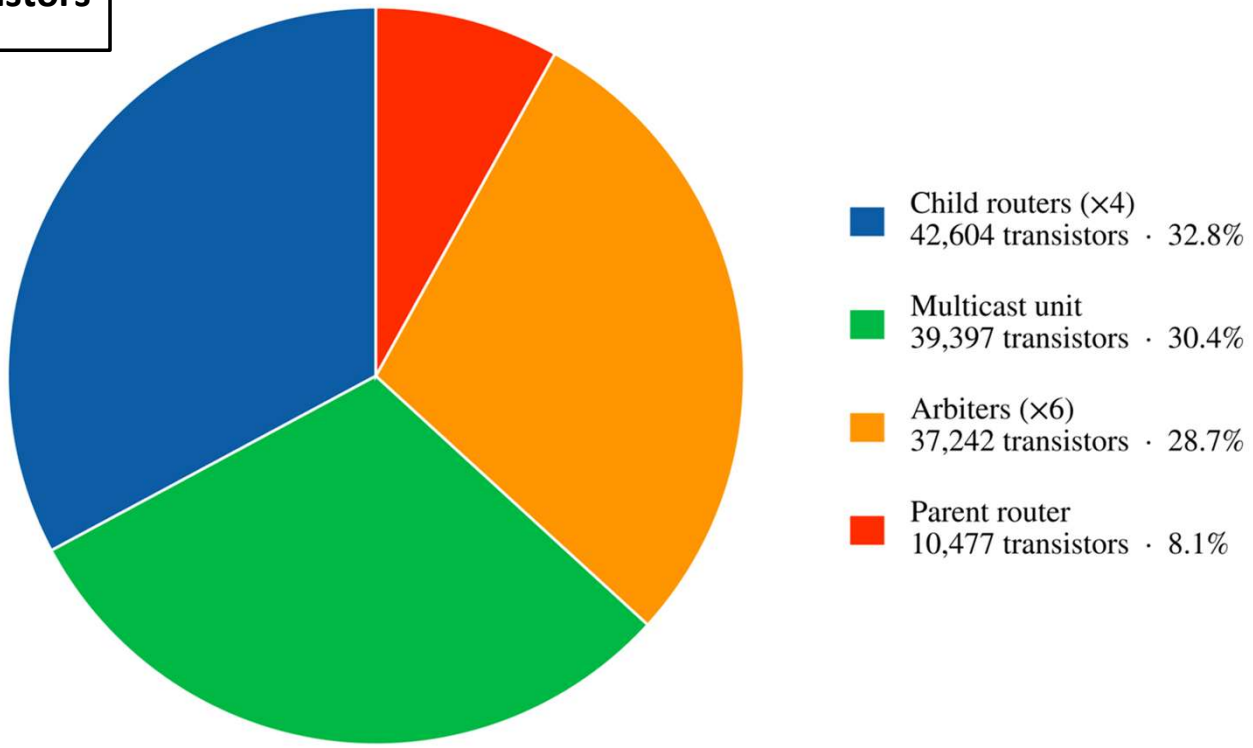
Energy consumptions mostly dependent on fanout

Energy per bit - Route parsing

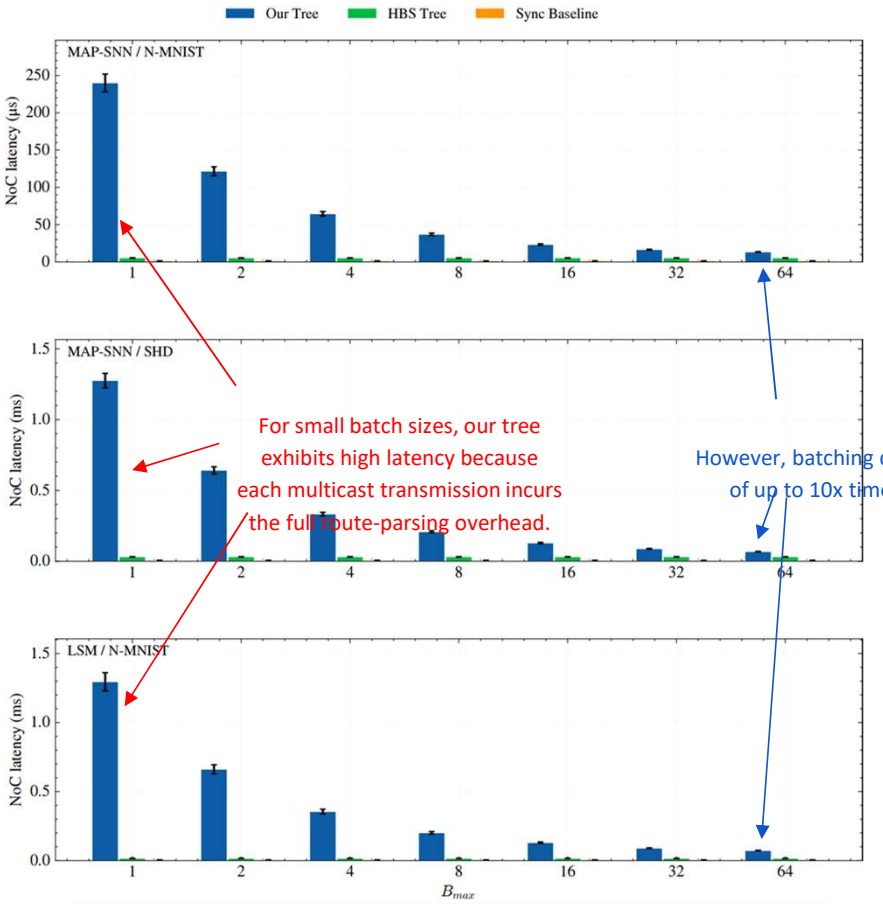
Parser phase	E [fJ/bit]
Initial route word	497.008–984.838
Continuation route word	343.314–923.297

Results - Router characterization

Total area: 129720 transistors



Results - Network benchmarks

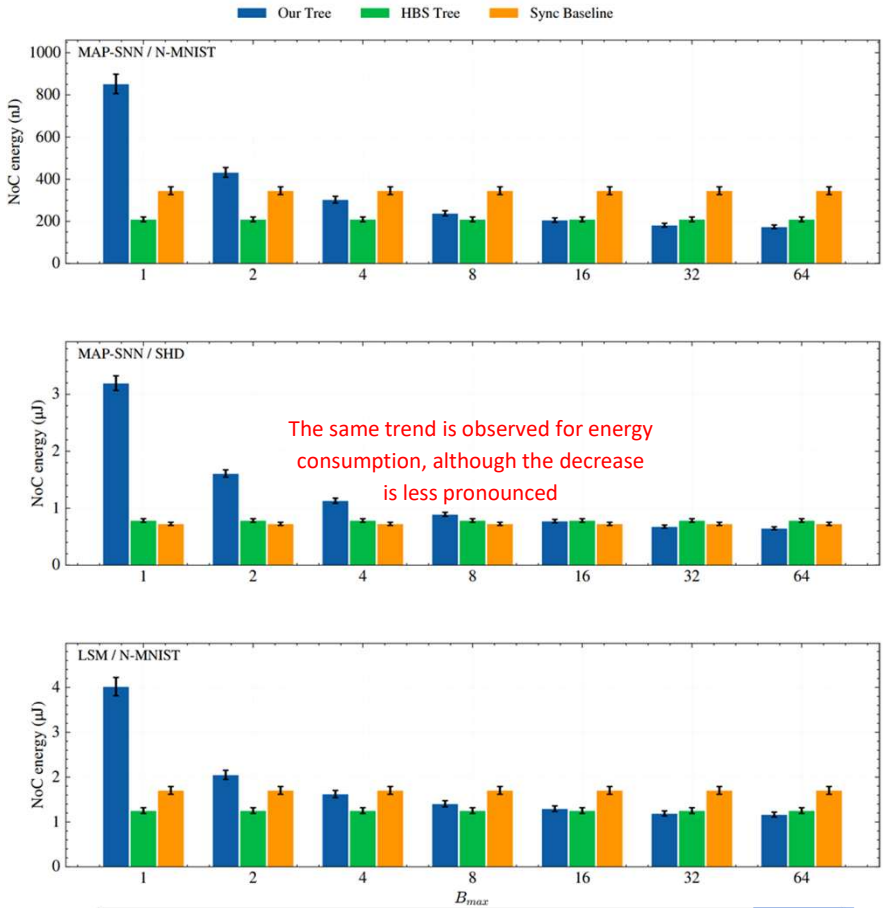


For small batch sizes, our tree exhibits high latency because each multicast transmission incurs the full route-parsing overhead.

However, batching decreases latency of up to 10x times for B_{max}=64

Workload	Our Tree	HBS Tree	Sync Baseline	ΔL
MAP-SNN / N-MNIST	13.5 μs	5.4 μs	1.36 μs	**150%**
MAP-SNN / SHD	68.1 μs	31.4 μs	6.2 μs	**117%**
LSM / N-MNIST	73.0 μs	15.9 μs	5.5 μs	**359%**

Still, our tree remains up to 3.6× slower than the HBS tree



The same trend is observed for energy consumption, although the decrease is less pronounced

Workload	Our Tree	HBS Tree	Sync Baseline	ΔE
MAP-SNN / N-MNIST	0.170 μJ	0.205 μJ	0.343 μJ	**17%**
MAP-SNN / SHD	0.64 μJ	0.78 μJ	0.71 μJ	**18%**
LSM / N-MNIST	1.15 μJ	1.24 μJ	1.70 μJ	**7%**

In this case, however, we observe an energy reduction of up to 18%.

Results - Network benchmarks

- A fully asynchronous quaternary-tree NoC with native exact multicast was implemented down to transistor level
  - Hierarchical routing represents only the **active multicast subtree**
  - Arbitrary destination sets with **no false-positive deliveries**
- Exact multicast becomes attractive when its routing overhead can be amortized
  - Batching allows multiple compatible source addresses to **share one multicast route**
  - Communication latency drops by **more than one order of magnitude** from B_{max}=1 to B_{max}=64
- Reducing unnecessary network activity translates into energy savings
  - At B_{max}=64 , dynamic NoC energy is reduced by **up to 18% VS HBS**
  - Despite our router being evaluated in **predictive 65 nm** versus a **post-layout 22 nm** HBS implementation
- Bit slicing, further PRS optimization, and a modern physical implementation provide substantial headroom for improvement

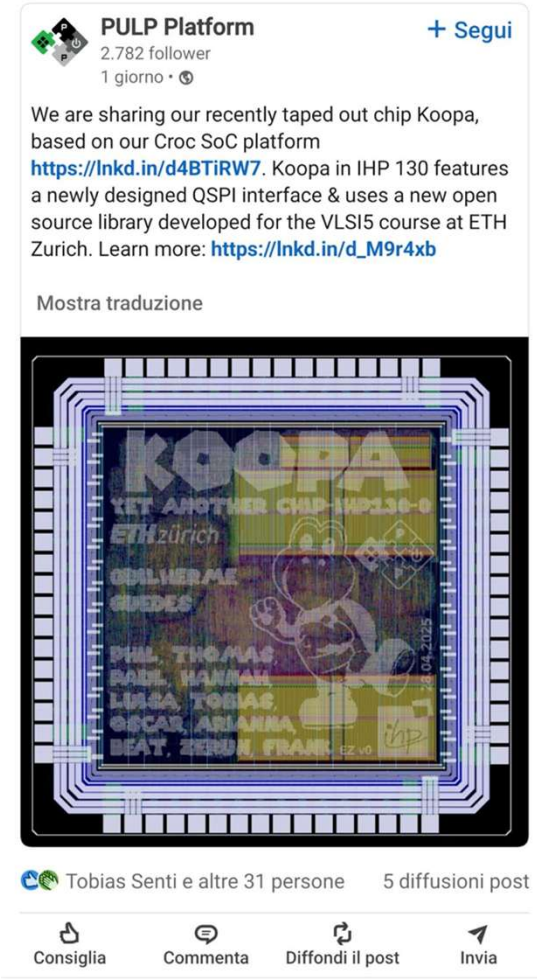
ETH zürich



Expanding the ez130 library

Expanding the ez130 library -Overview

- **Context**
  - **VLSI5:** first time the course was offered. Very limited slots.
  - **Scope:** expand an existing open source 130nm library of PULP
- **Goal**
  - Design 6 new standard cells
- **Workflow**
  - **Schematic design:** design exploration, logic restructuring
  - **Stick diagram:** share diffusions, optimize connections
  - **Physical design:** Cadence
  - **Characterization:** PEX, generate .lef and .gds
  - **Post-layout verification:** implemented a small design that used them



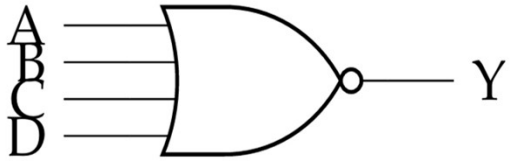
Our cells are already getting used

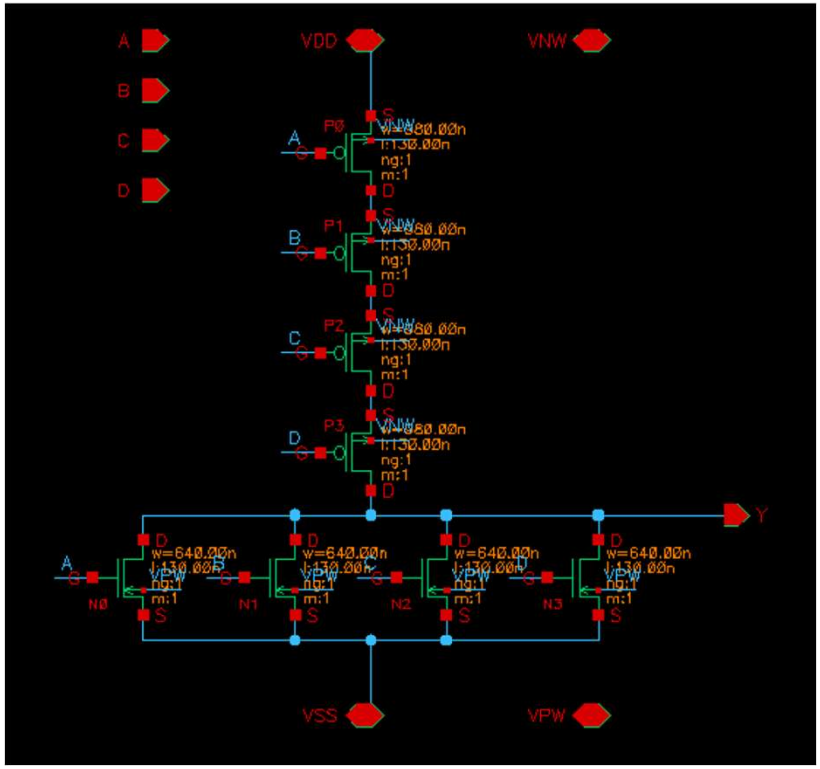
Expanding the ez130 library -NOR4X2 / X1

Logic function: $\overline{A + B + C + D}$

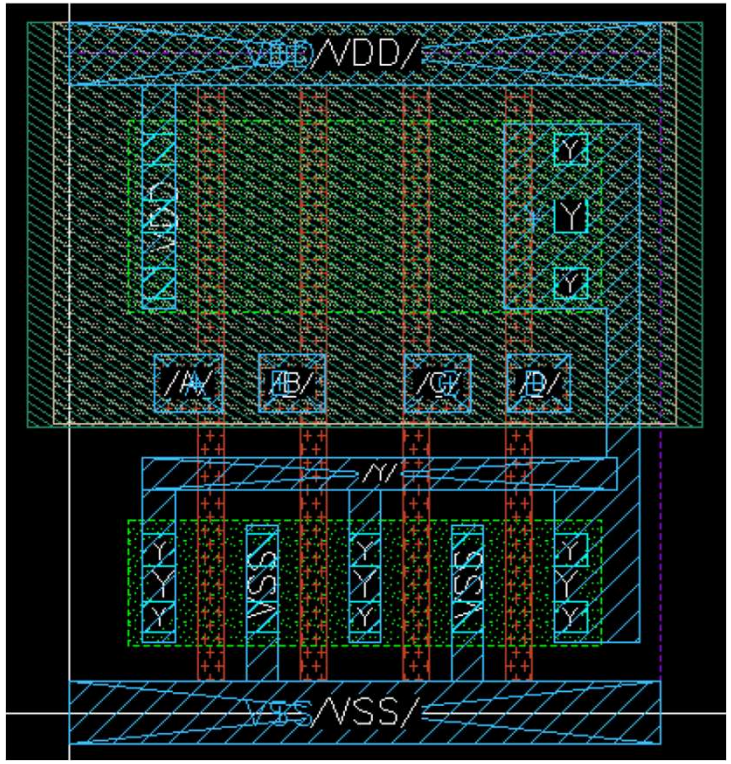
N° of transistors: 8

Area: 9.8784 µm²





NOR4X2 schematic



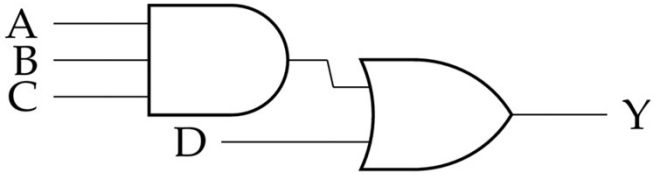
NOR4X2 layout

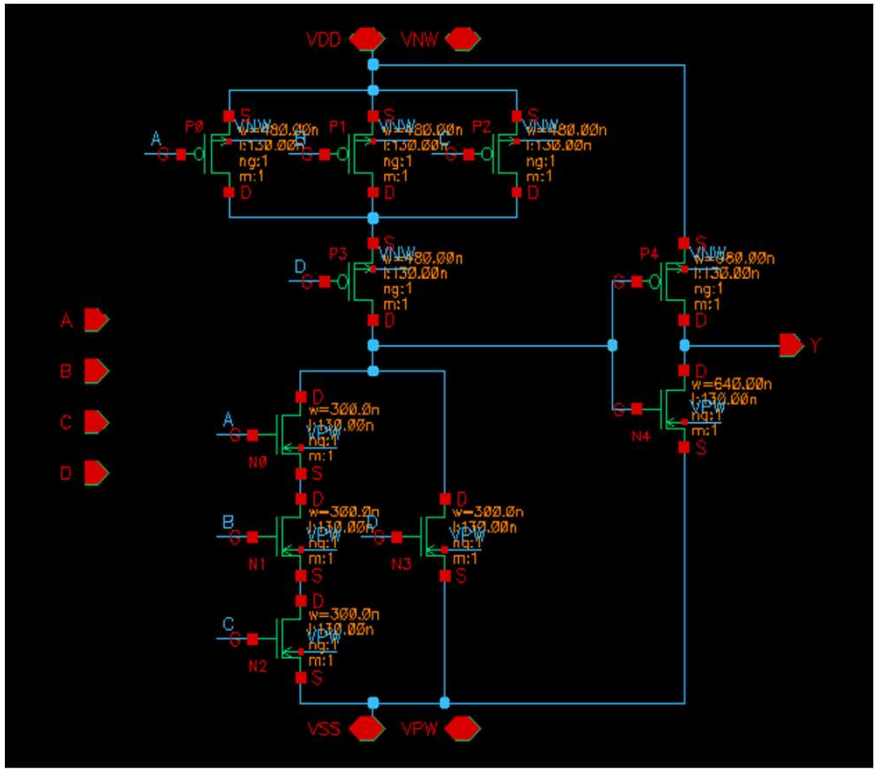
Expanding the ez130 library –AO31X2 / X1

Logic function: $A \cdot B \cdot C + D$

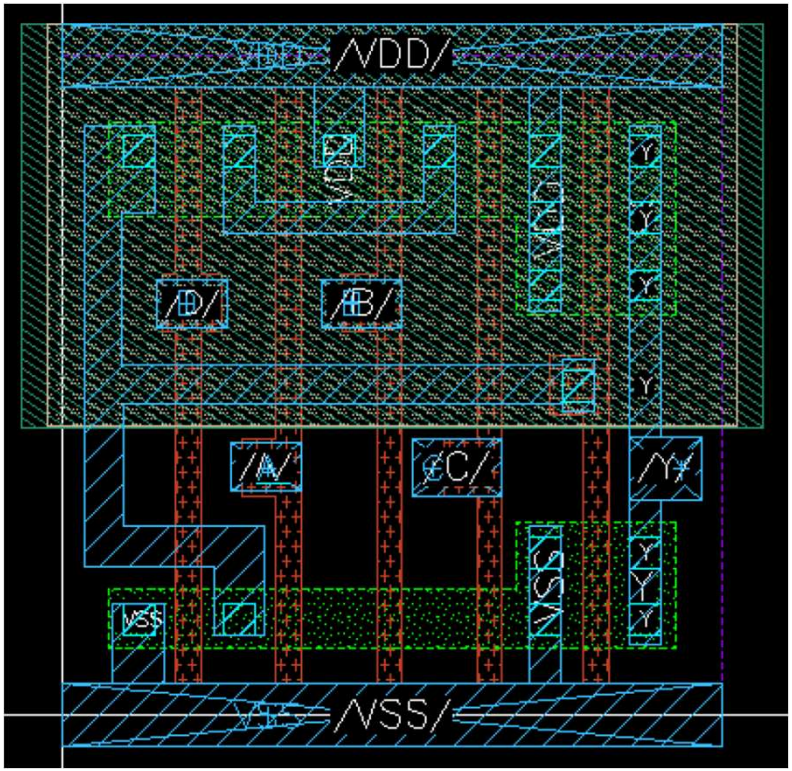
N° of transistors: 10

Area: 11.2896 µm²





AO31X2 schematic



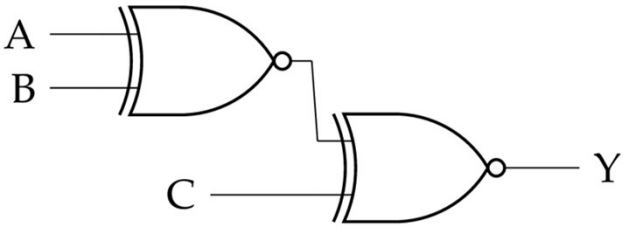
AO31X2 layout

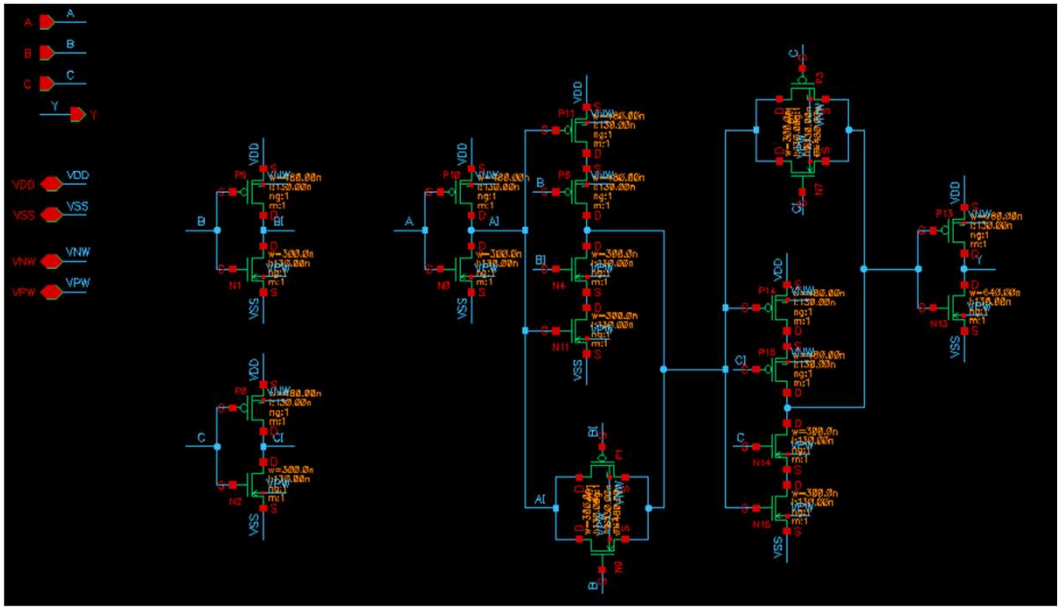
Expanding the ez130 library –XNOR3X2 / X1

Logic function: $\overline{A \oplus B \oplus C}$

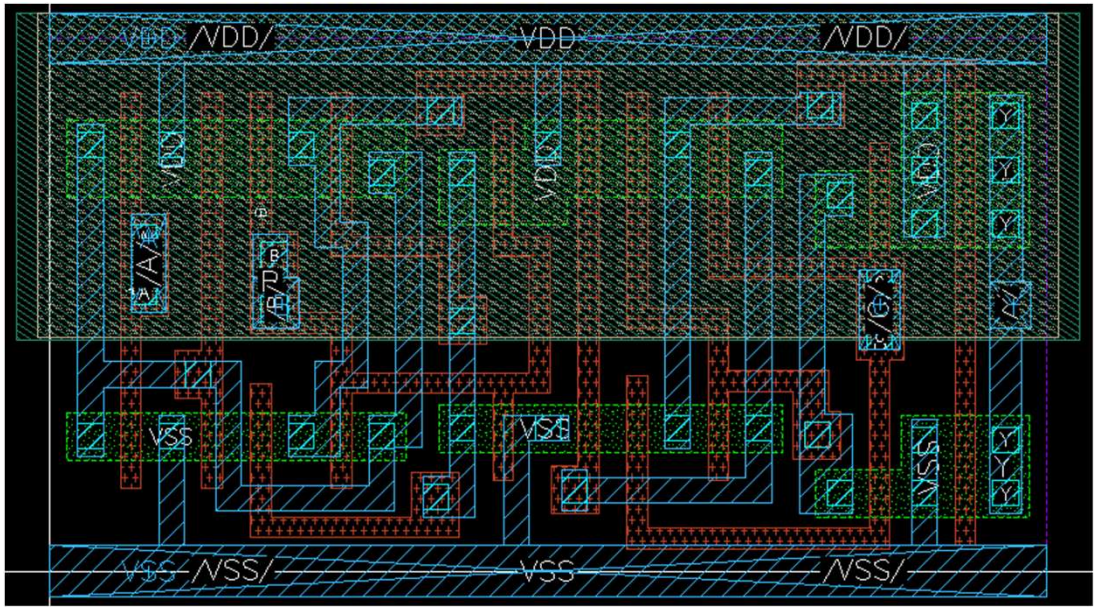
N° of transistors: 20

Area: 21.168 µm²



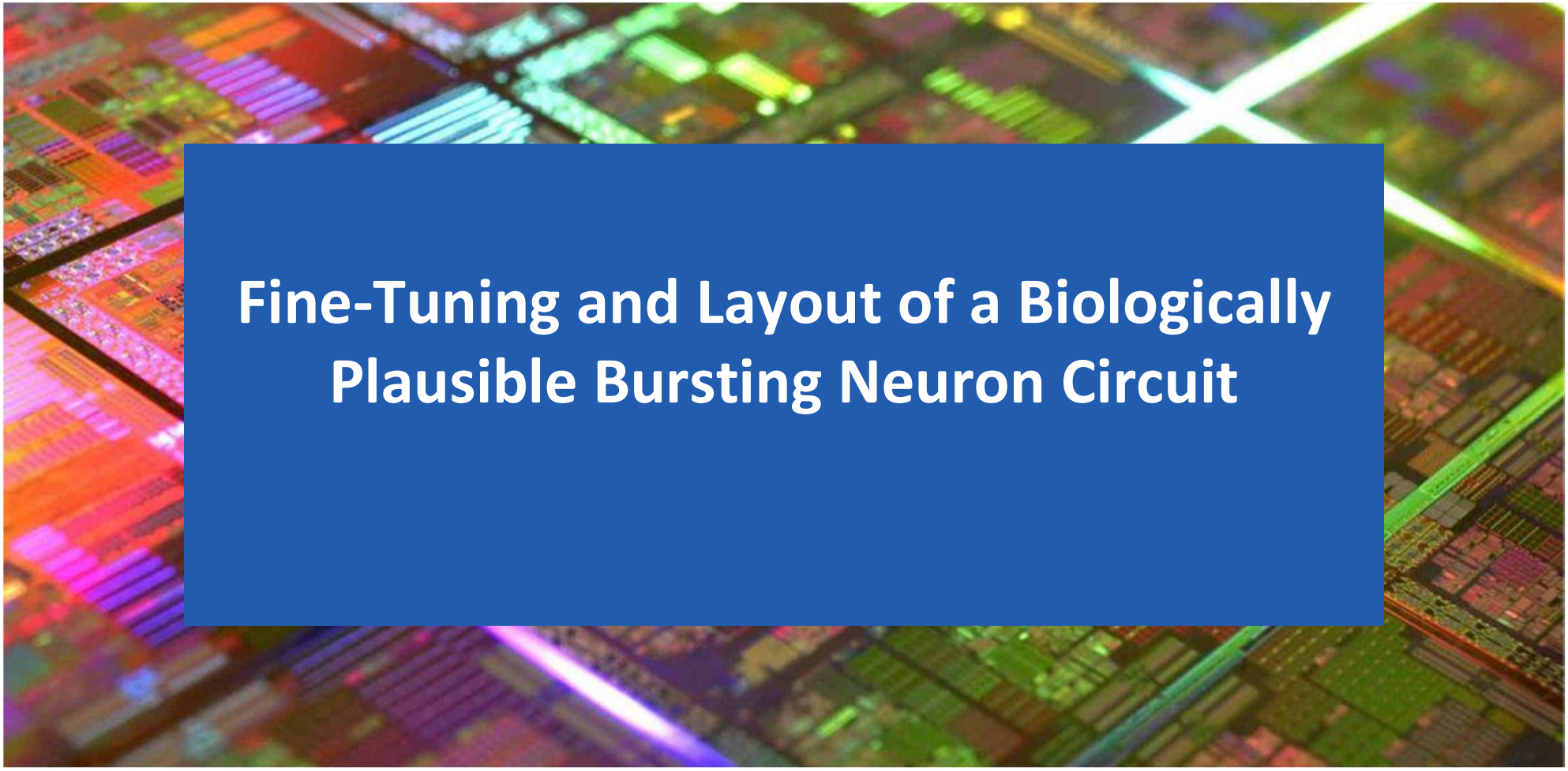


XNOR3X2 schematic



XNOR3X2 layout

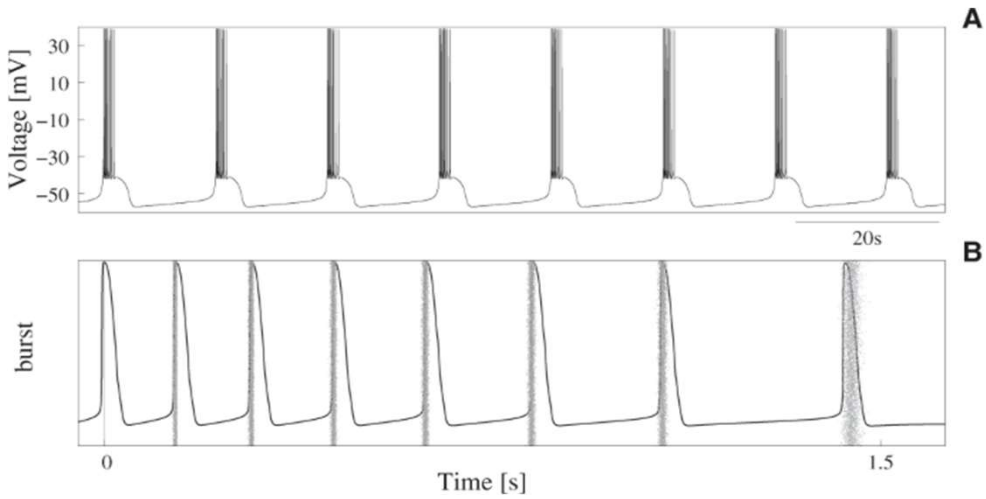
ETH zürich



Fine-Tuning and Layout of a Biologically Plausible Bursting Neuron Circuit

Bursting Neuron -Overview

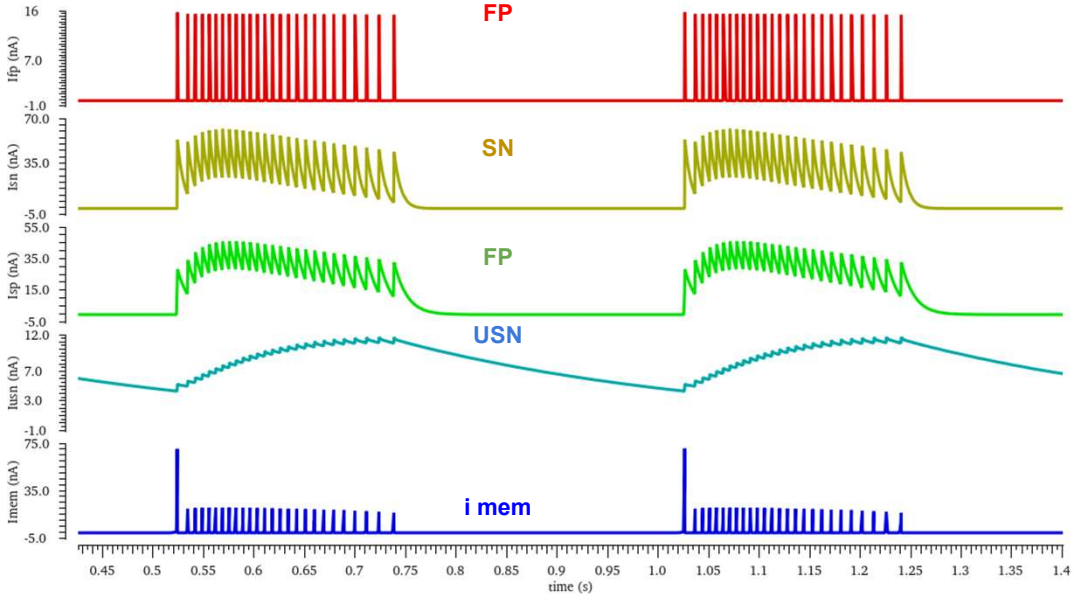
- **Context**
  - **Neuromorphic II:** analog IC design
  - **Scope:** learn of to design the layout of analog circuit
- **Goal**
  - Obtain a functioning layout of the given circuit
- **Workflow**
  - **Tuning:** find the right parameters and transistor sizes
  - **Simulations:** make sure the spiking activity is happening
  - **Physical implementation:** layout of the circuit
  - **Post-layout simulations:** Montecarlo simulations



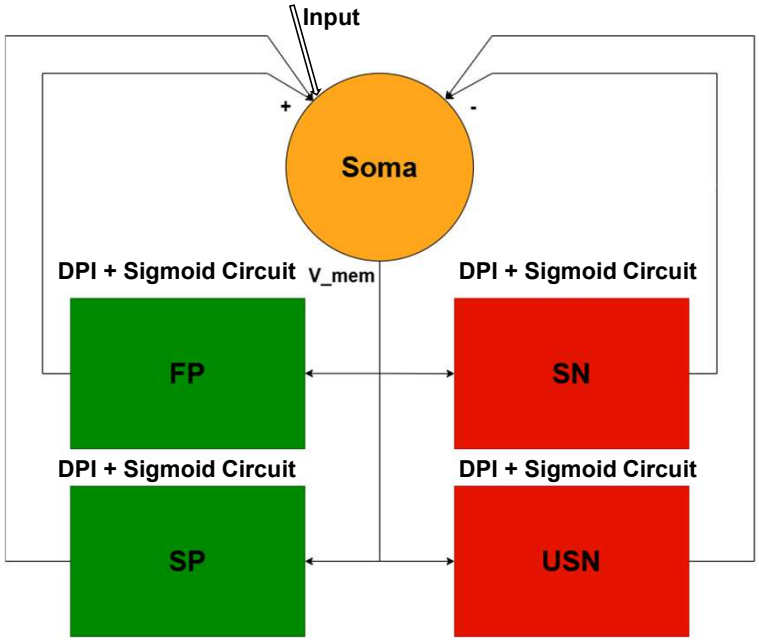
Bursting activity in biological neurons

Bursting Neuron –Circuit overview

- **Input: DC current**
- **Output: bursts in membrane current**
- **4 Feedback mechanisms (DPI + Sigmoid):**
  - FP (fast positive), SN (slow negative), SP (slow positive), USN (ultra slow negative)

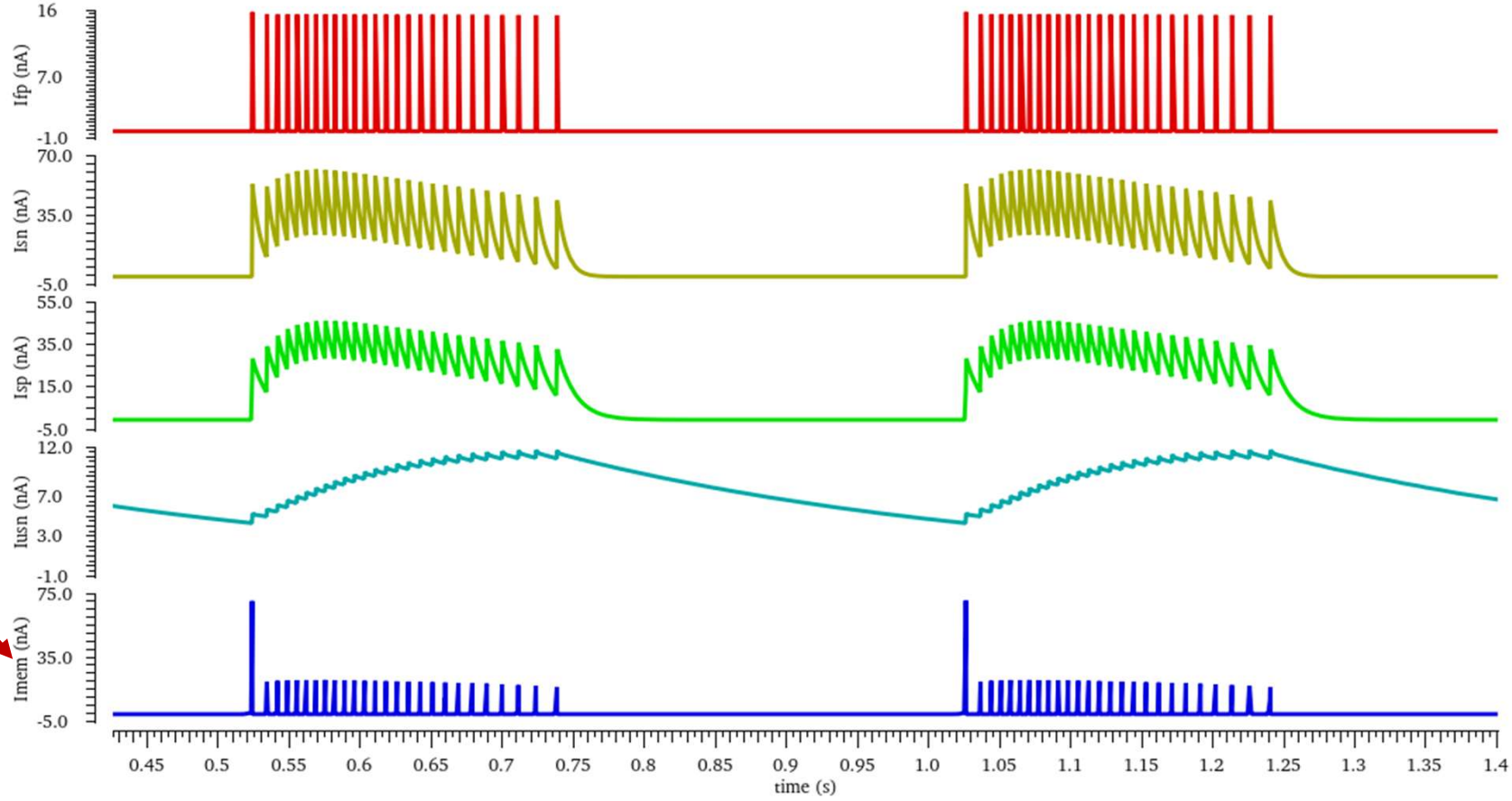


Bursting activity in our circuit



Model used

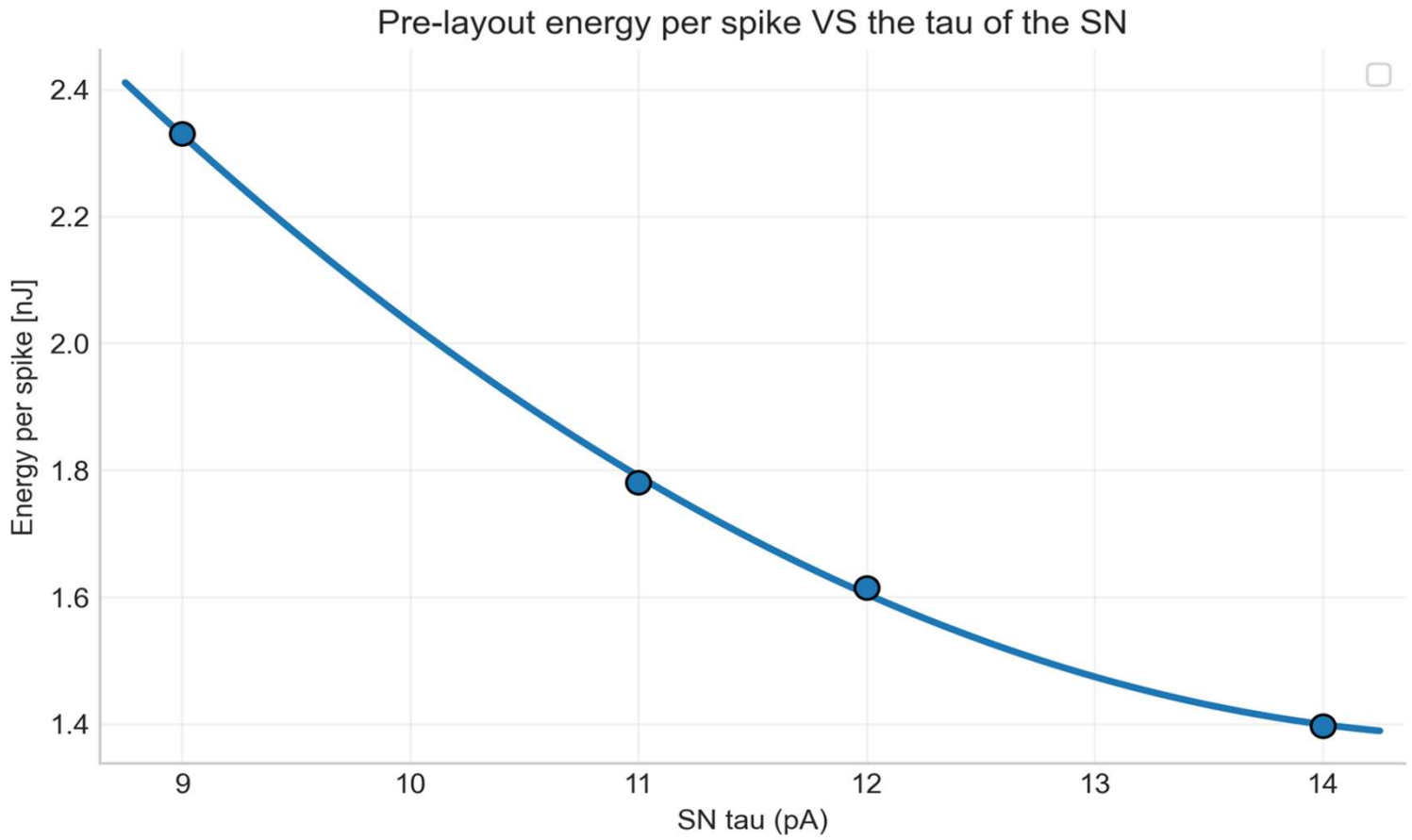
Bursting Neuron –Bursting activity



Bursting activity in our circuit

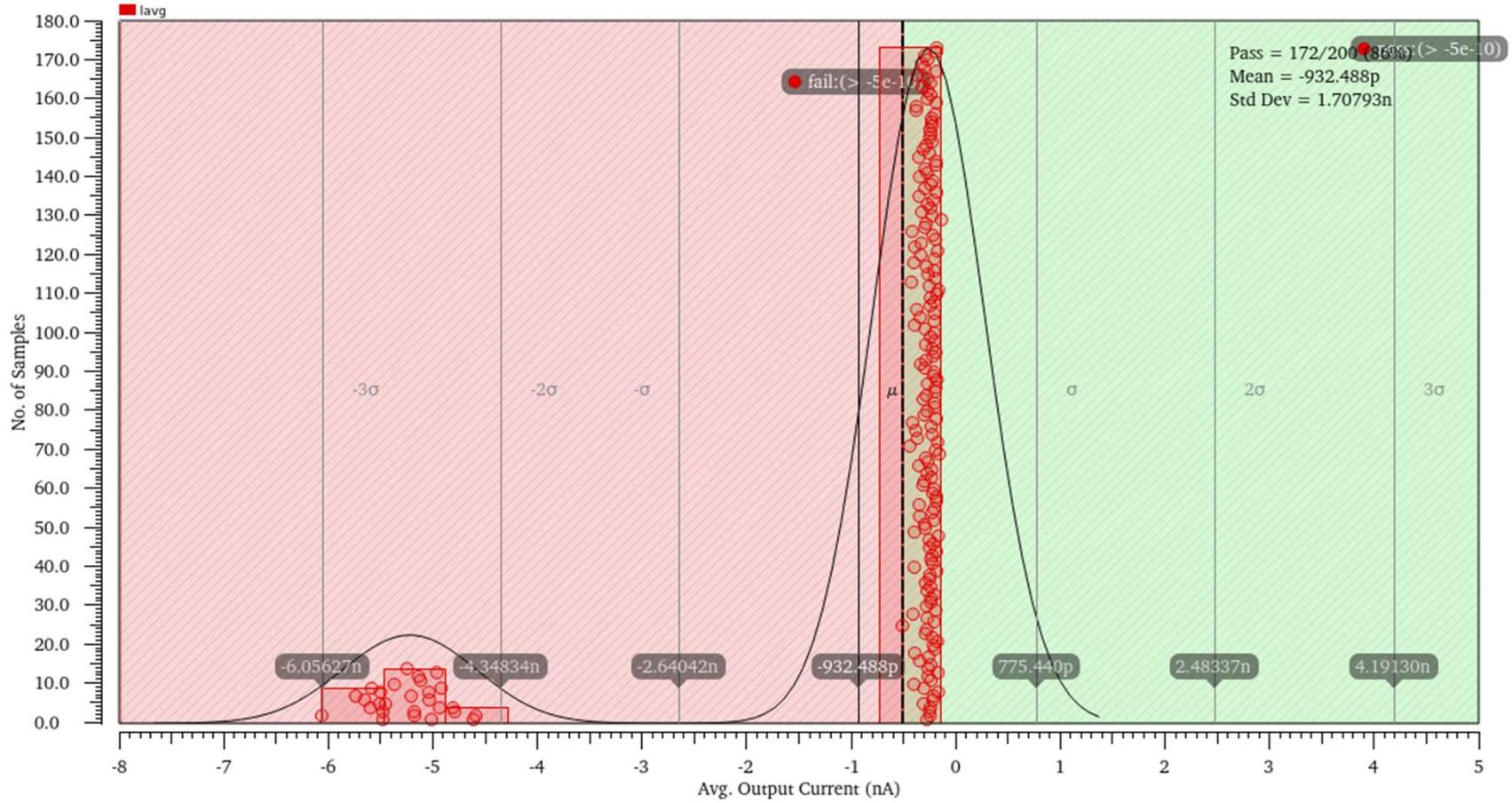
Bursting Neuron –Power analysis Pre-layout

The energy consumption for each spike decreases with the tau of the SN since it makes the spikes more narrow.



Bursting Neuron –Montecarlo Pre-layout

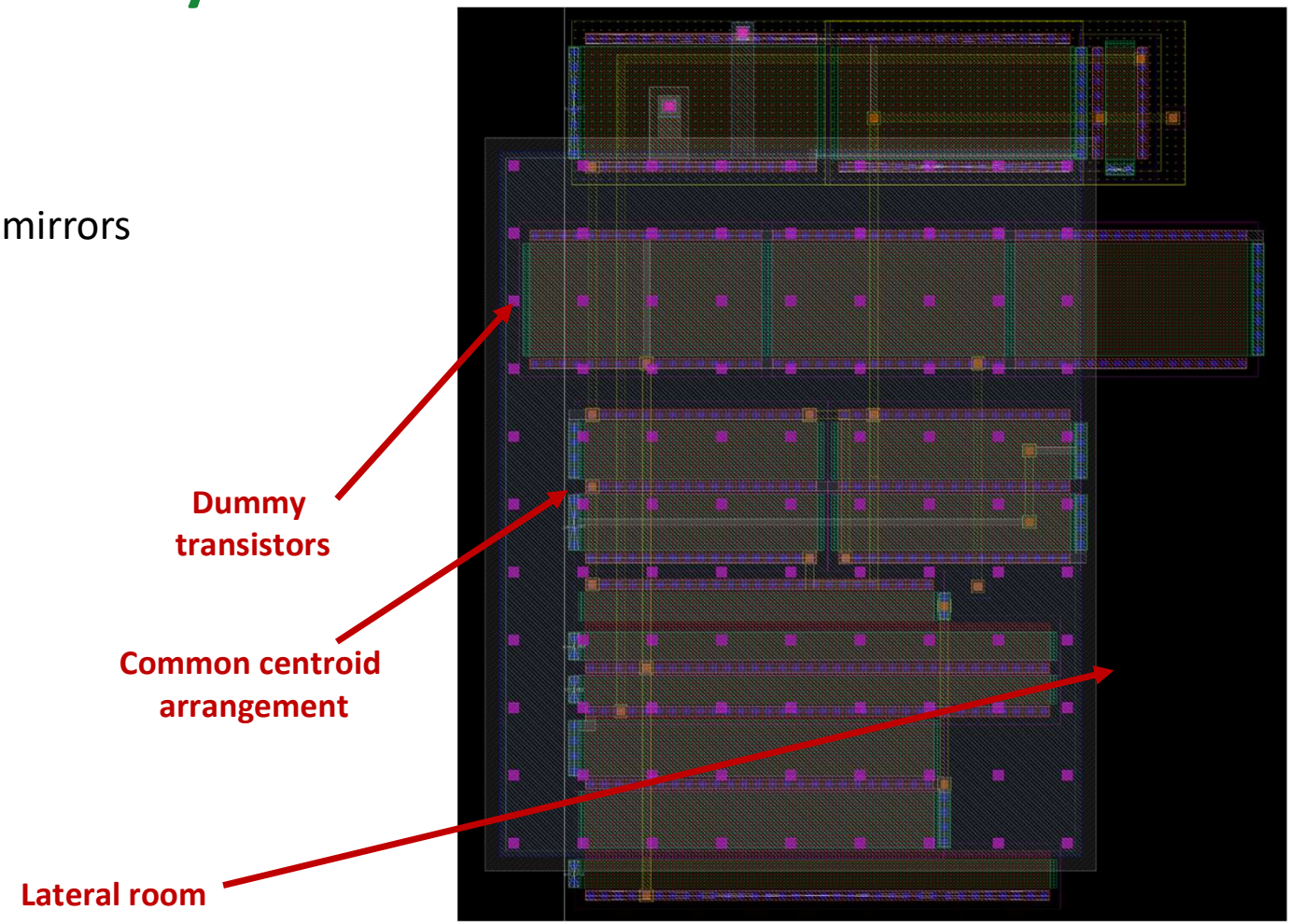
Passing samples: 86%



Montecarlo simulation results for mismatch + process variations

Bursting Neuron –Feedback layout

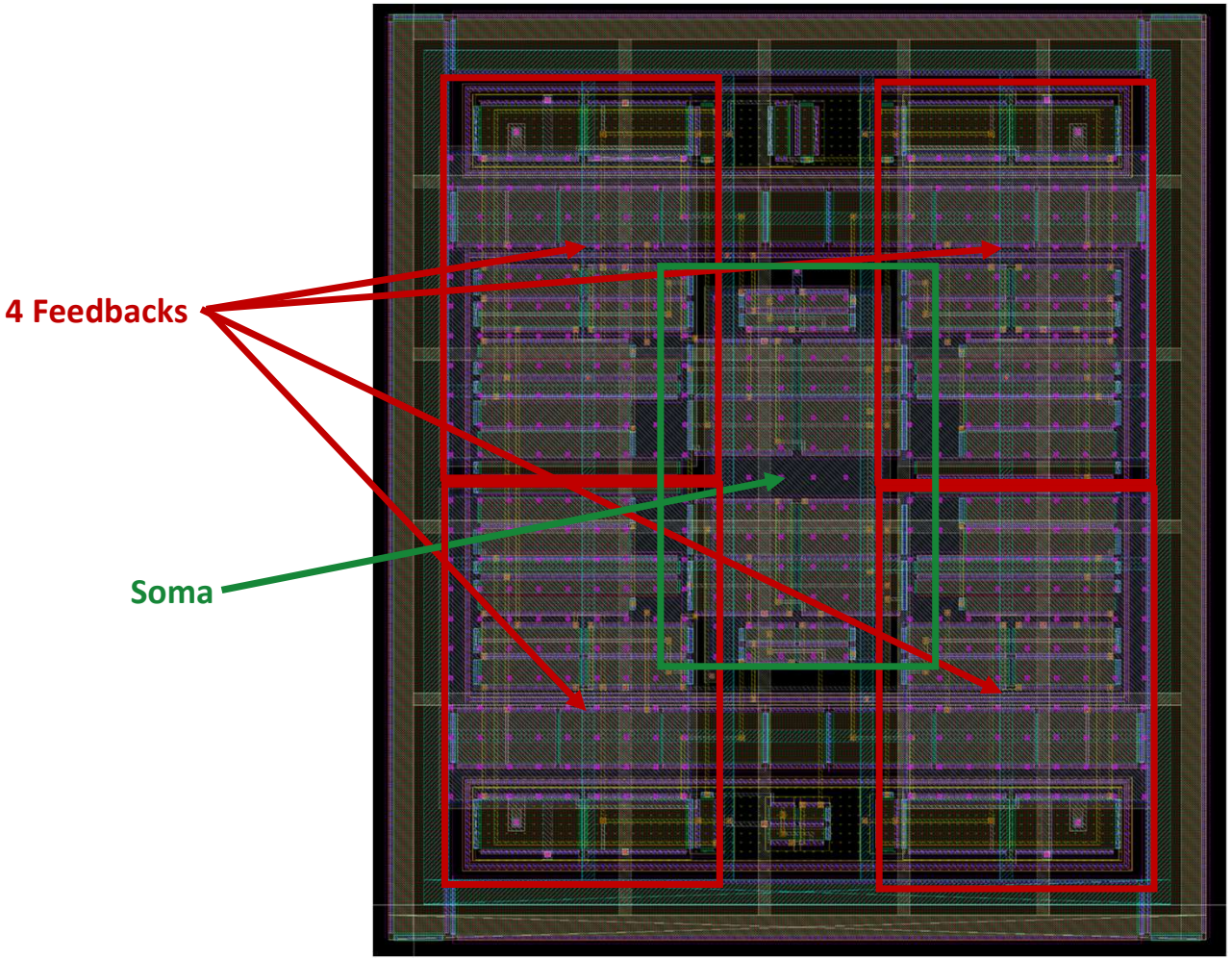
- **4 copies of the feedback block**
- **Features:**
  - Centroid arrangement for current mirrors
  - Stripe of dummy transistor
  - Lateral room for the Soma's block



Layout of one of the feedback mechanisms

Bursting Neuron –Final layout

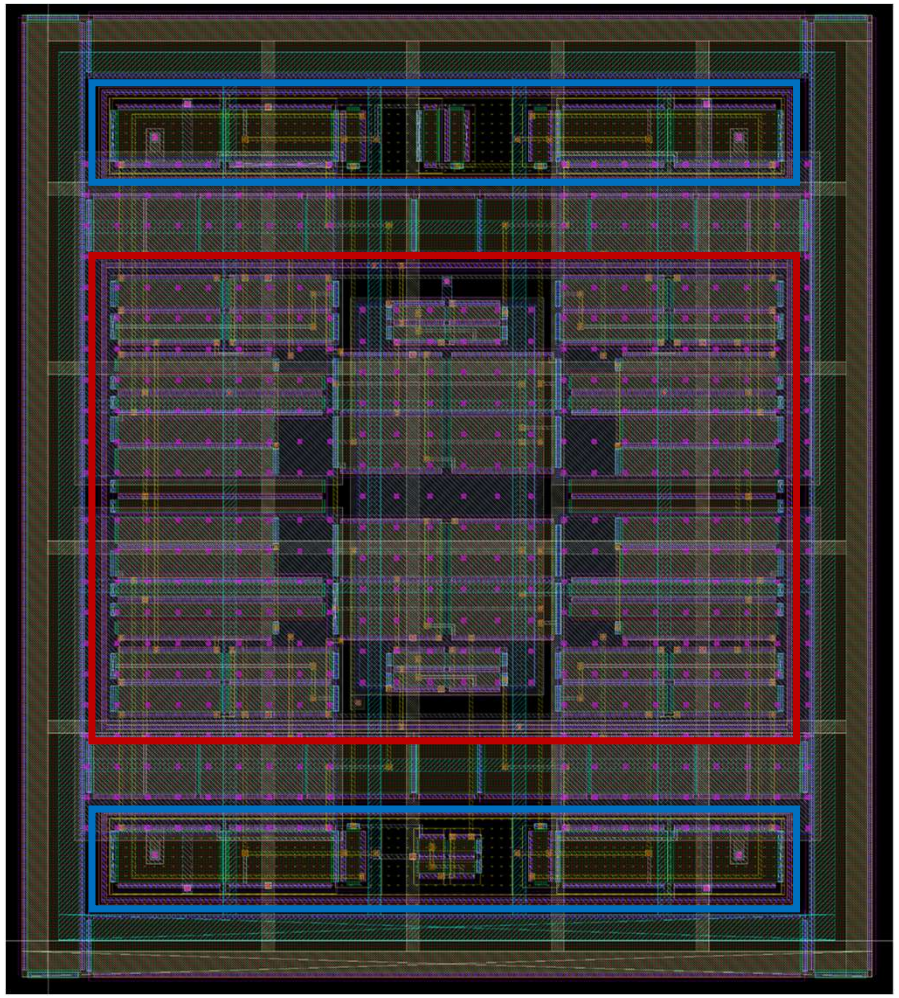
- **Structure**
  - 4 feedback blocks
  - Soma in the center
  - Two rows of PMOS
  - Central block of NMOS
- **Guard rings:**
  - P guard rings around NMOS
  - N guard rings around PMOS
- **Dummy transistors**
  - To separate PMOS and NMOS
  - All around
- **Power grid**
- **Area: 75.66 × 67.02 µm²**



Layout of the final neuron

Bursting Neuron –Final layout

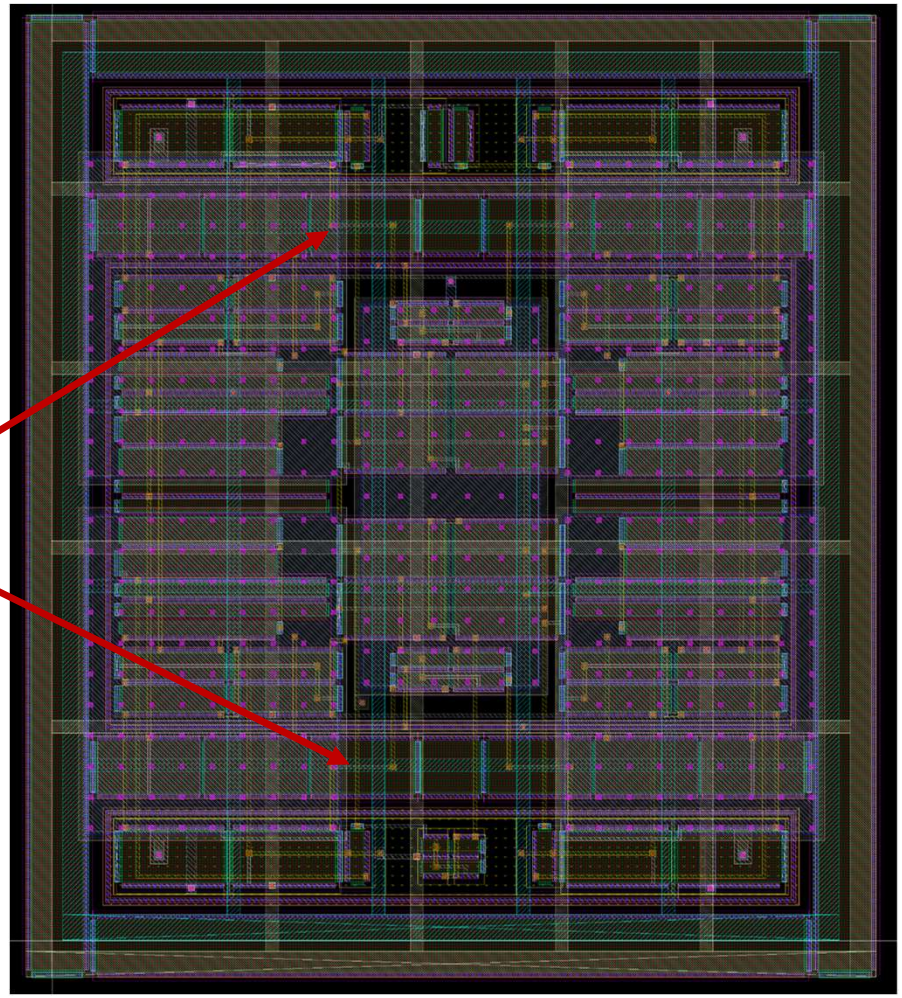
- **Structure**
  - 4 feedback blocks
  - Soma in the center
  - Two rows of PMOS
  - Central block of NMOS
- **Guard rings:**
  - P guard rings around NMOS
  - N guard rings around PMOS
- **Dummy transistors**
  - To separate PMOS and NMOS
  - All around
- **Power grid**
- **Area: 75.66 × 67.02 µm²**



Layout of the final neuron

Bursting Neuron –Final layout

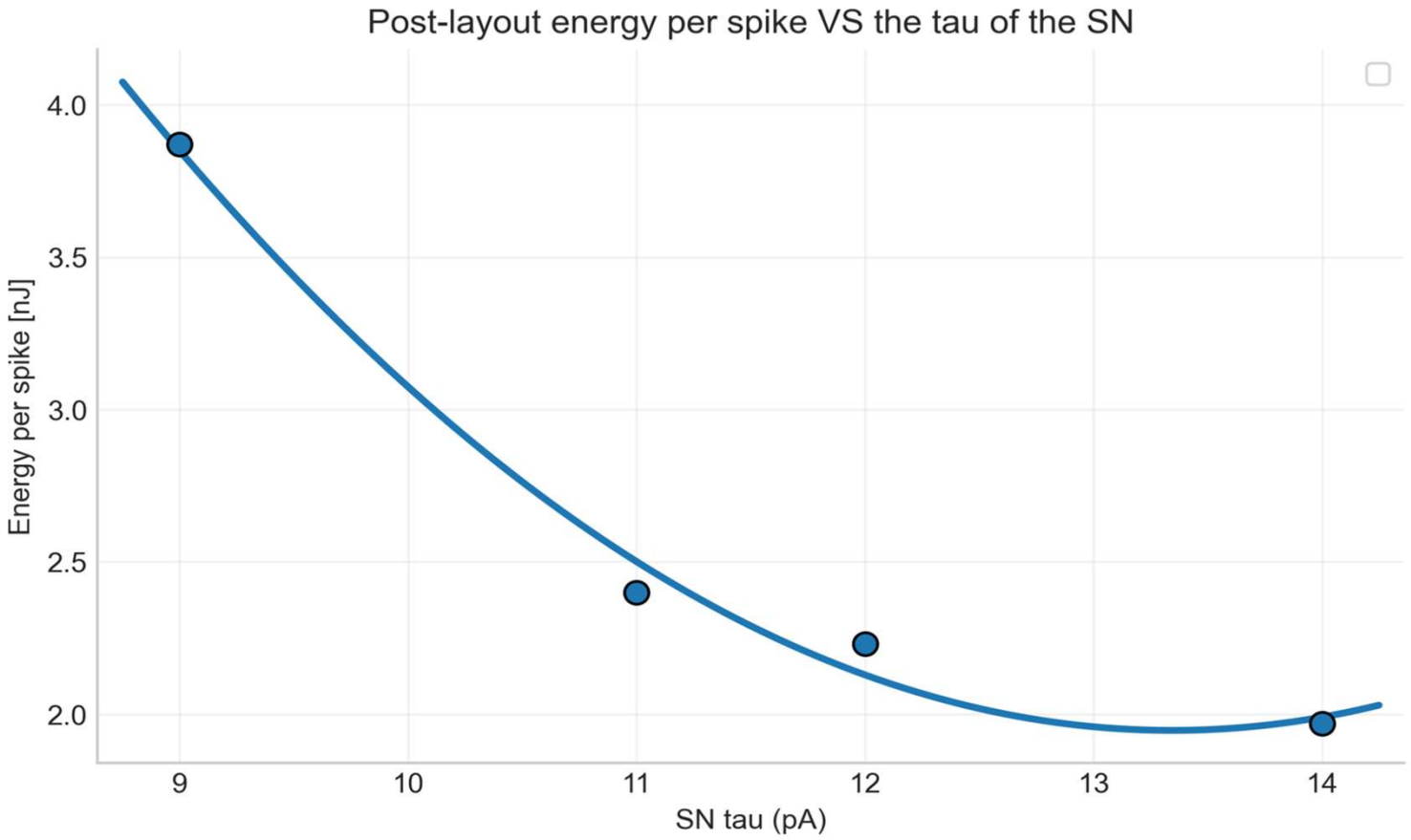
- **Structure**
  - 4 feedback blocks
  - Soma in the center
  - Two rows of PMOS
  - Central block of NMOS
- **Guard rings:**
  - P guard rings around NMOS
  - N guard rings around PMOS
- **Dummy transistors**
  - To separate PMOS and NMOS
  - All around
- **Power grid**
- **Area: 75.66 × 67.02 µm²**



Layout of the final neuron

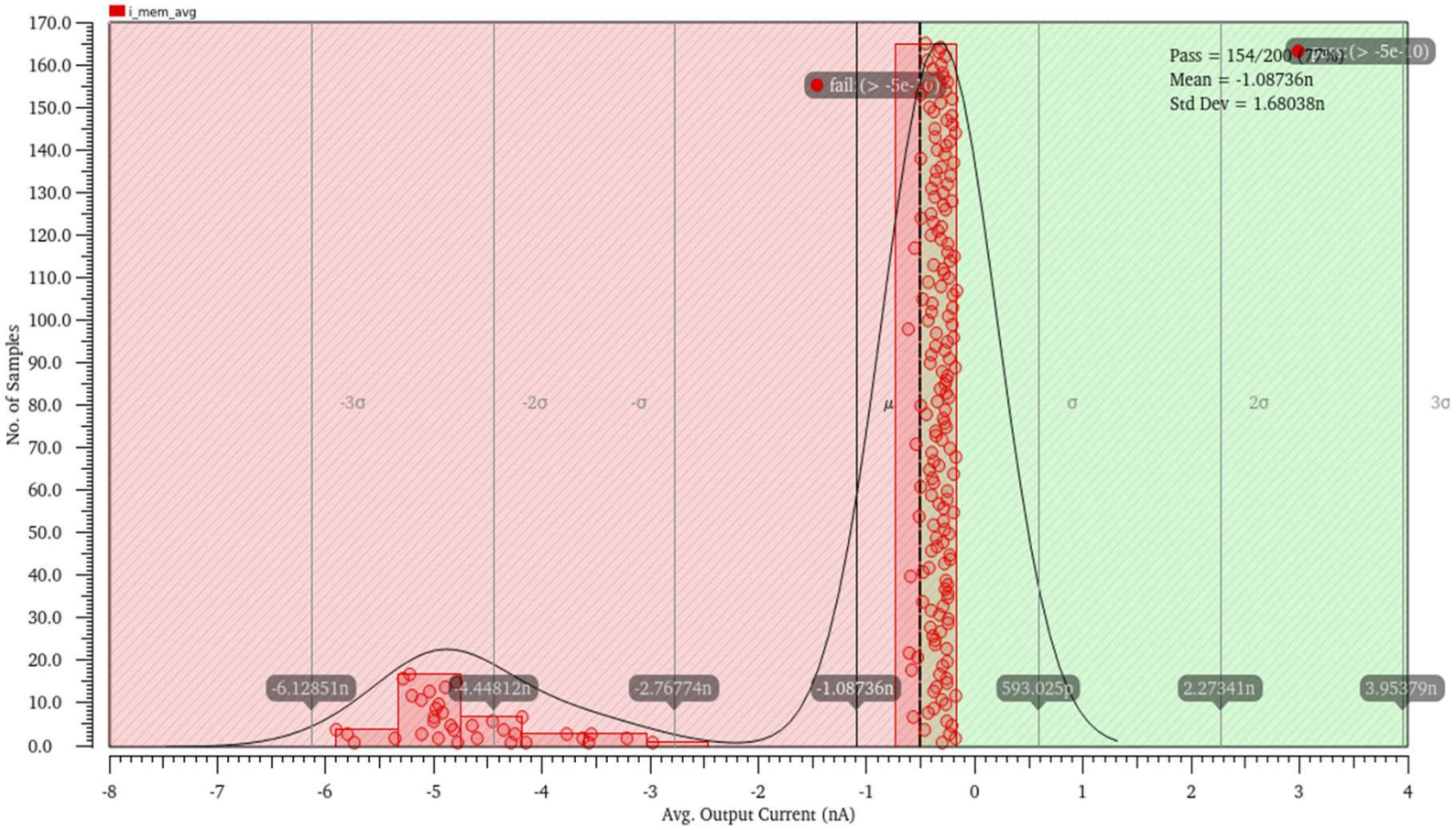
Bursting Neuron –Power analysis Post-layout

As expected, the power consumption is higher post layout, due to the fact that now parasitics are taken into account as well.



Bursting Neuron –Montecarlo Post-layout

Passing samples: 77%



Montecarlo simulation results for mismatch + process variations

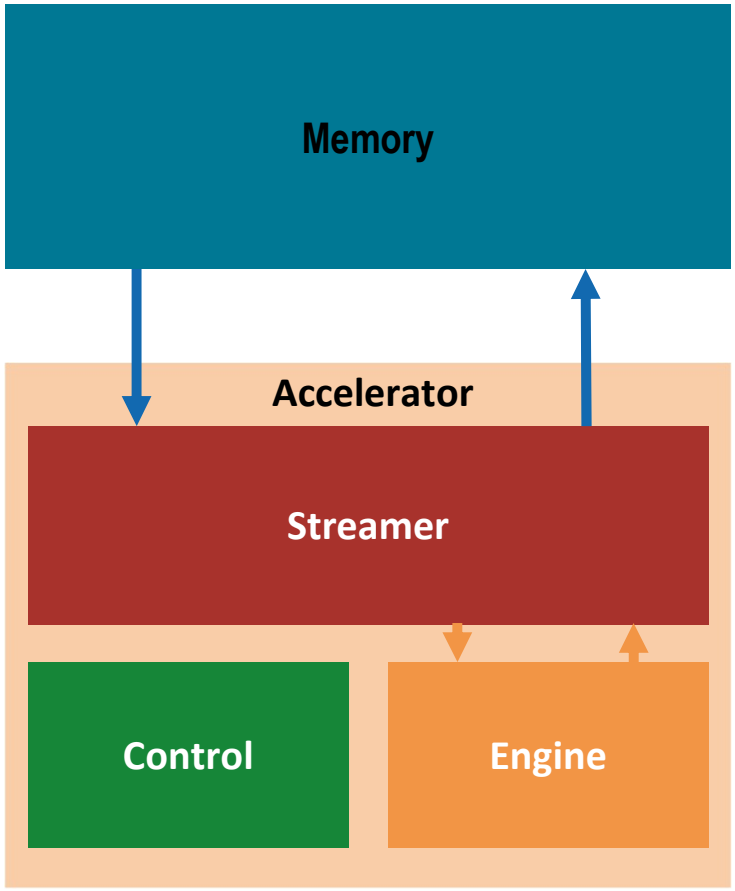
ETH zürich



Designing a SpMM accelerator

SpMM Accelerator-Overview

- **Context**
  - **Semester thesis:** under Institute for Integrated Systems (IIS)
  - **Scope:** learn the higher-end of an ASIC workflow
- **Goal**
  - Have the functioning RTL of the sparsity-aware streamer of the accelerator
- **Workflow**
  - **Analytical evaluation:** Gustavson and Outer product dataflows
  - **Architecture design:** design the architecture of the streamer
  - **Functional verification:** different matrix sizes and sparsity
  - **Synthesis:** area results



Structure of the accelerator

Case study

- **Assumptions on data**
  - X sparse
  - Y dense
- **X encoded with bitmap format**
  - The matrix is stored as its dense elements + a one-hot encoded mask of metadata
- **The engine takes chunks of the matrices**
  - The size of the engine determines the size of the tiles of X and Y.



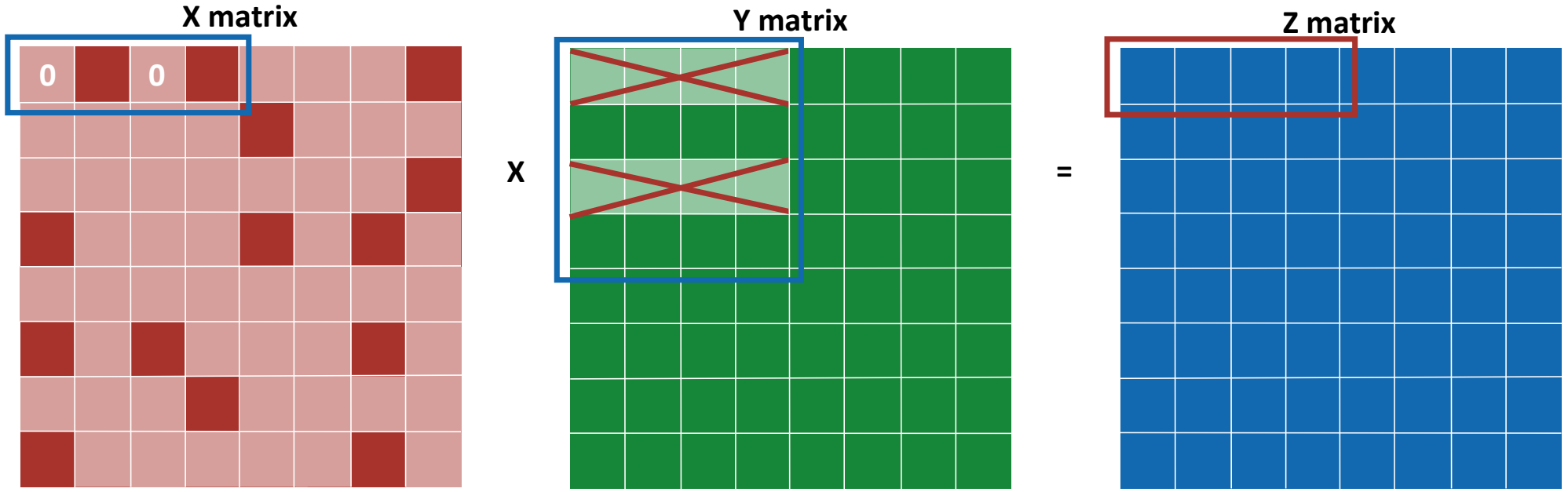
Naming conventions used

My contributions

- **Analytical evaluation**
  - Compared Gustavson vs. Outer Product dataflows under various reuse strategies for Sparse Matrix Multiplication (SpMM)
- **Sparsity-aware extension of the Hardware Processing Engine's (HWPE) streamer**
  - Integrated sparsity handling logic into the streamer
  - Functionally verified on arbitrary matrix, varying sizes and sparsity levels
- **Design Evaluation**
  - Performance
  - Area overhead

The idea

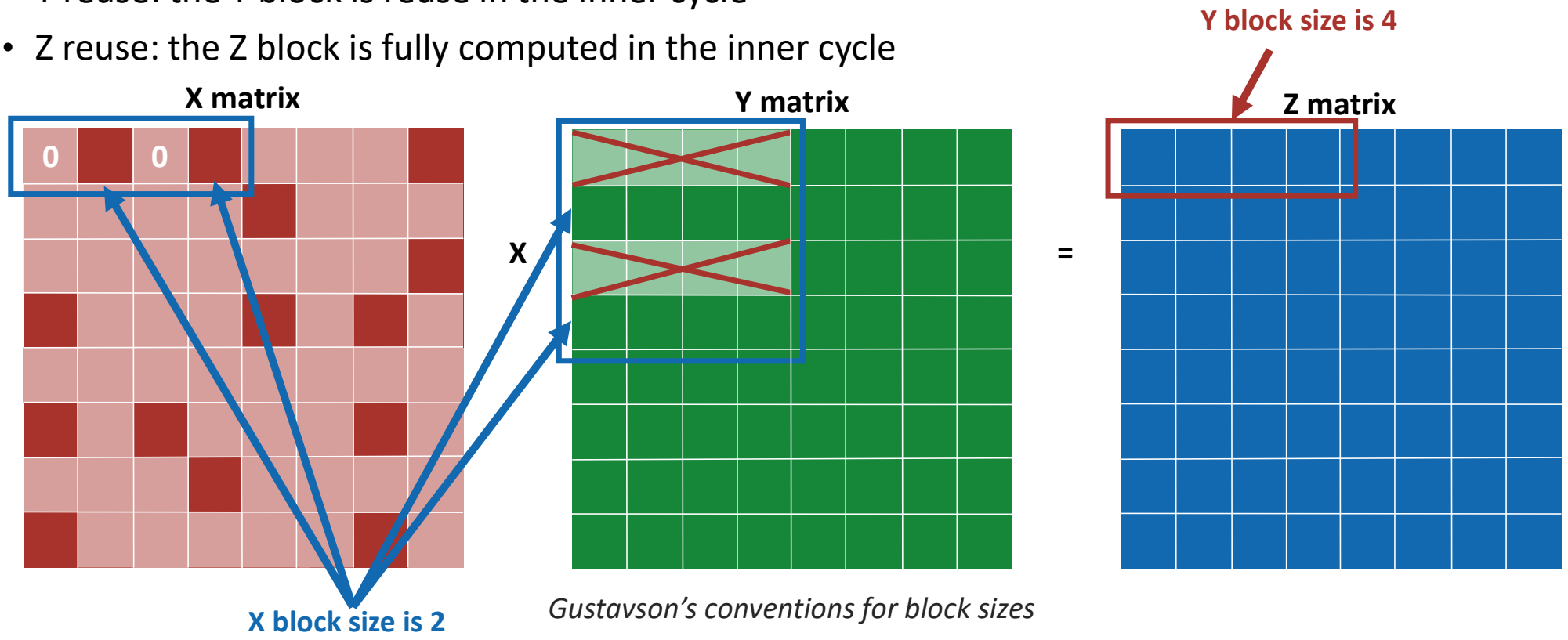
- **Avoid multiplications by zero**
  - If the element in the row of X is a zero, I can skip the correspondent row of Y
  - This way I avoid unnecessary loads and multiplications that consume bandwidth and cycles



Example with Gustavson

Dataflows options -Gustavson

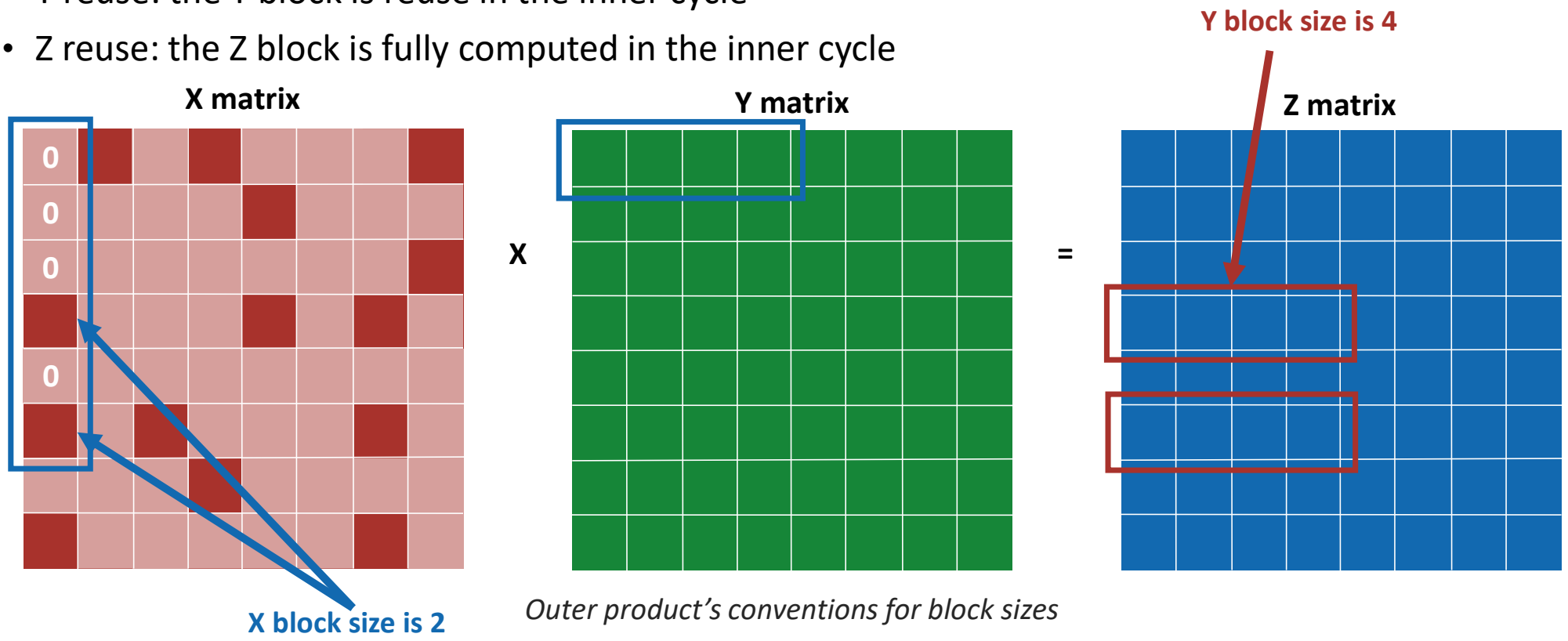
- **There are 3 possible options**
  - X reuse: the X block is reused in the inner cycle
  - Y reuse: the Y block is reuse in the inner cycle
  - Z reuse: the Z block is fully computed in the inner cycle



Gustavson's conventions for block sizes

Dataflows options –Outer product

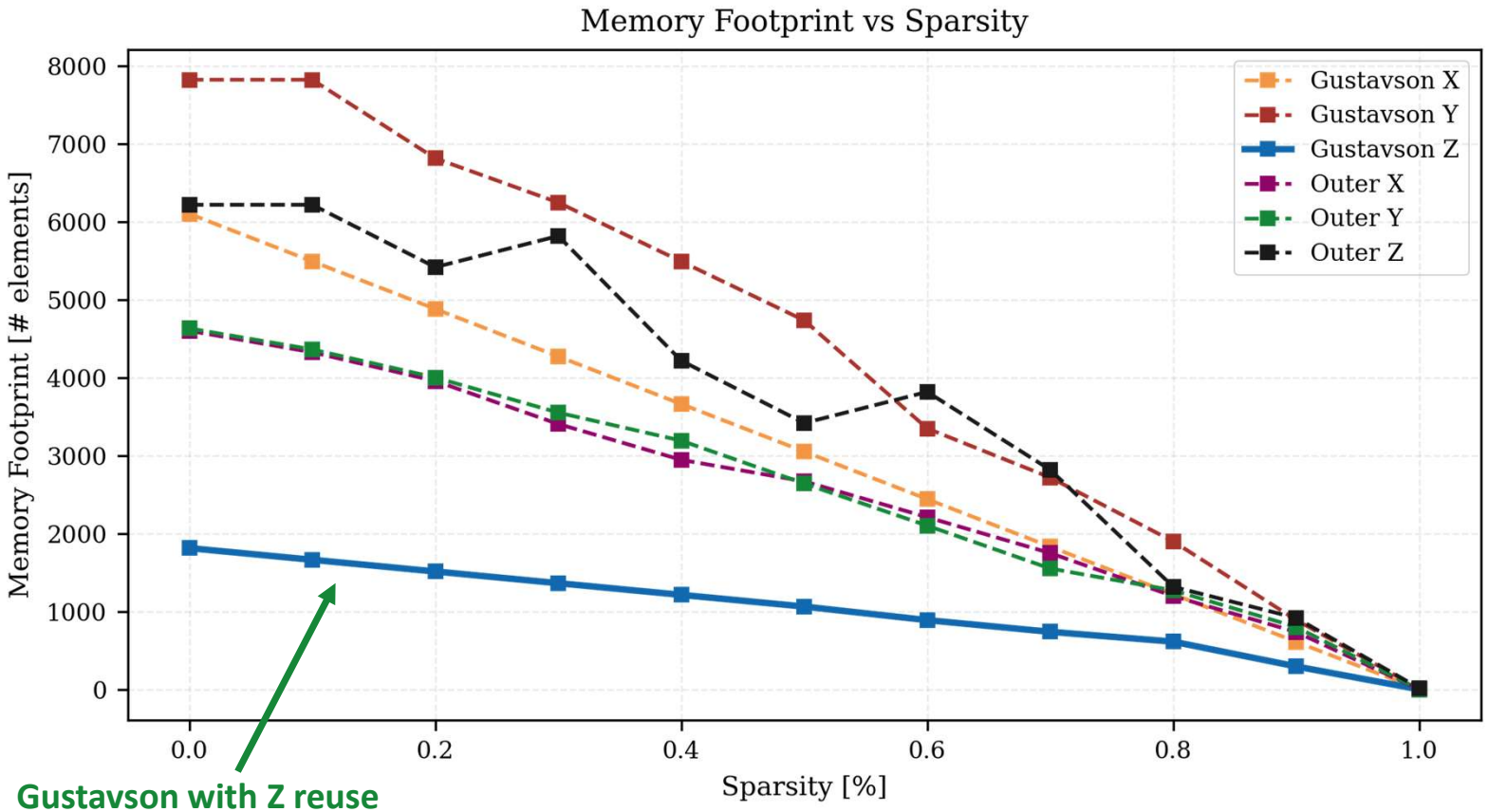
- **There are 3 possible options**
  - X reuse: the X block is reused in the inner cycle
  - Y reuse: the Y block is reuse in the inner cycle
  - Z reuse: the Z block is fully computed in the inner cycle



Outer product's conventions for block sizes

Analytical models comparison –Memory footprint

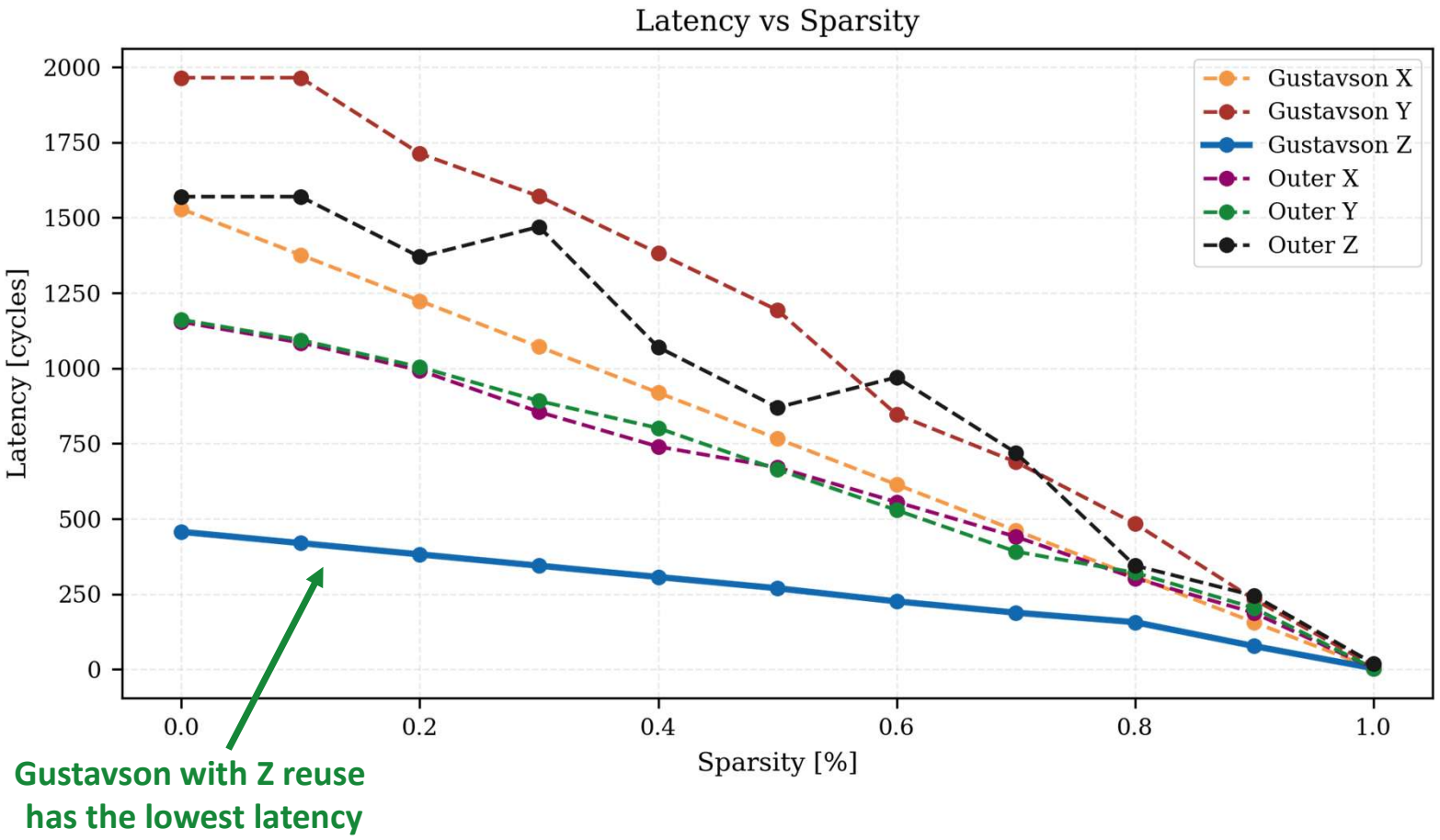
- **Assumptions:**
  - MAC units = 4
  - X rows = 32
  - Y columns = 32
  - Inner dim. = 32
  - Metadata chunk = 128b
  - Input data size = 32b
  - Output data size = 64b
  - BW = 128b



Gustavson with Z reuse has the lowest memory footprint

Analytical models comparison –Latency

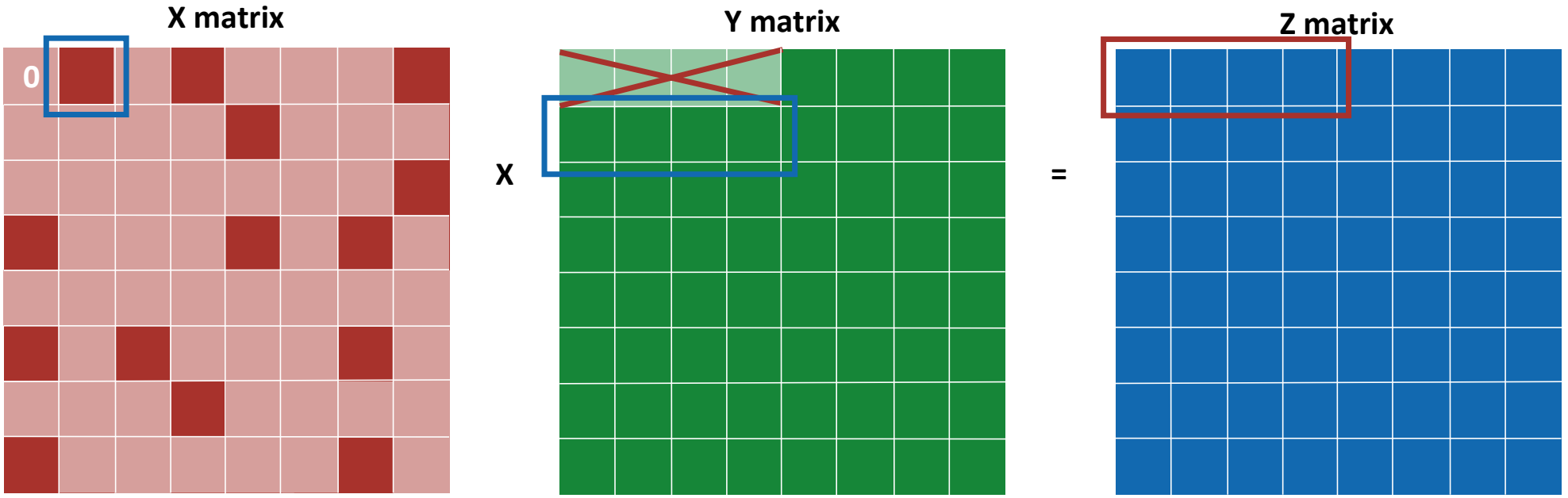
- **Assumptions:**
  - MAC units = 4
  - X rows = 32
  - Y columns = 32
  - Inner dim. = 32
  - Metadata chunk = 128b
  - Input data size = 32b
  - Output data size = 64b
  - BW = 128b



Gustavson with Z reuse has the lowest latency

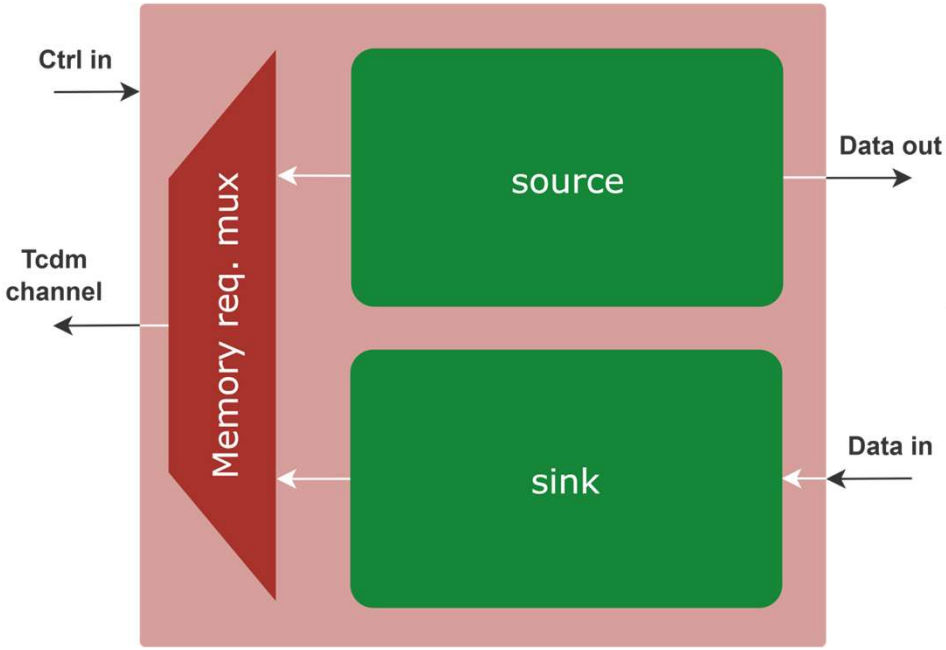
Final choice

- **Overall the Gustavson dataflow with Z reuse seems to be the best**
  - Never loads chunks of Z from memory
  - Reuses the metadata as much as possible



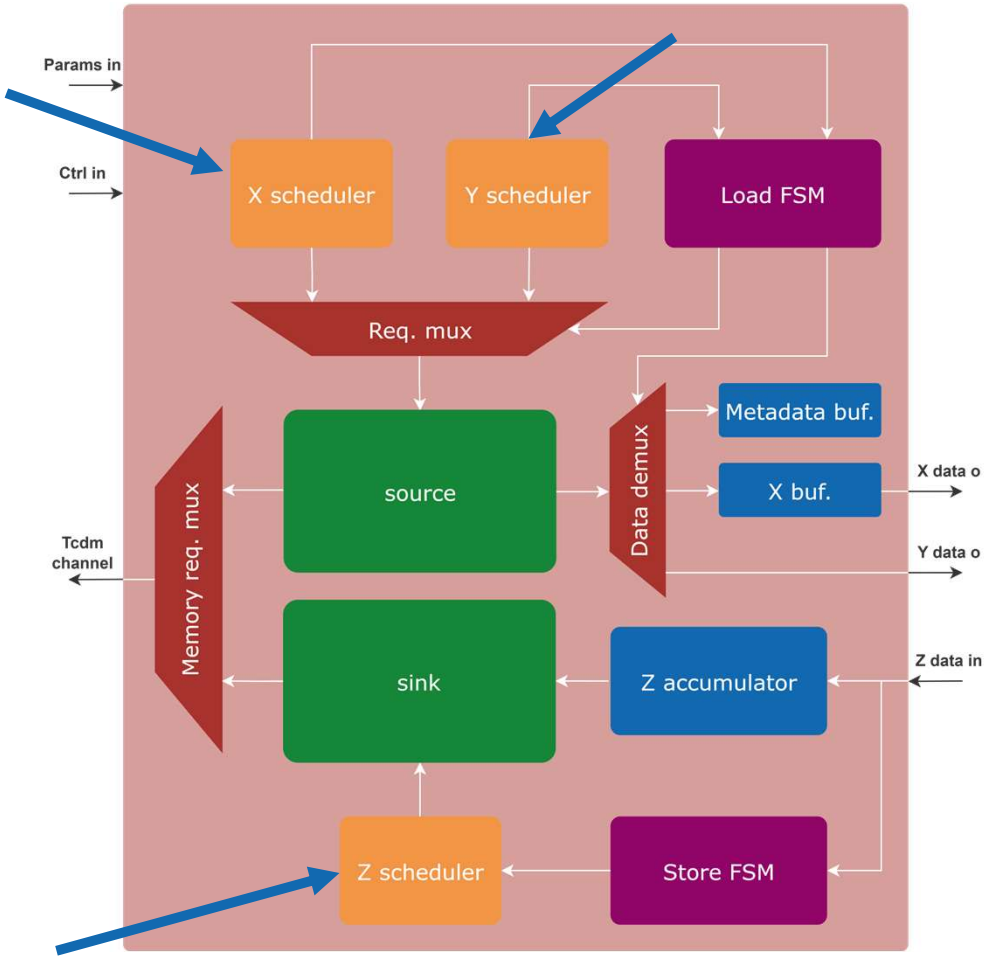
Gustavson dataflow with Z stationary

Accelerator's streamer architecture VS datamover one



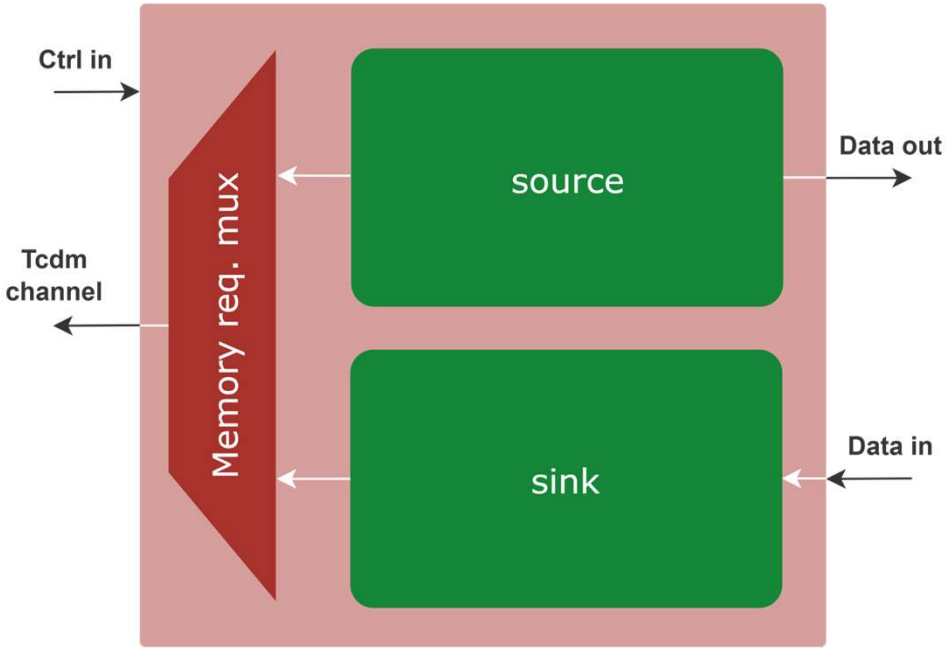
Datamover's streamer architecture

VS



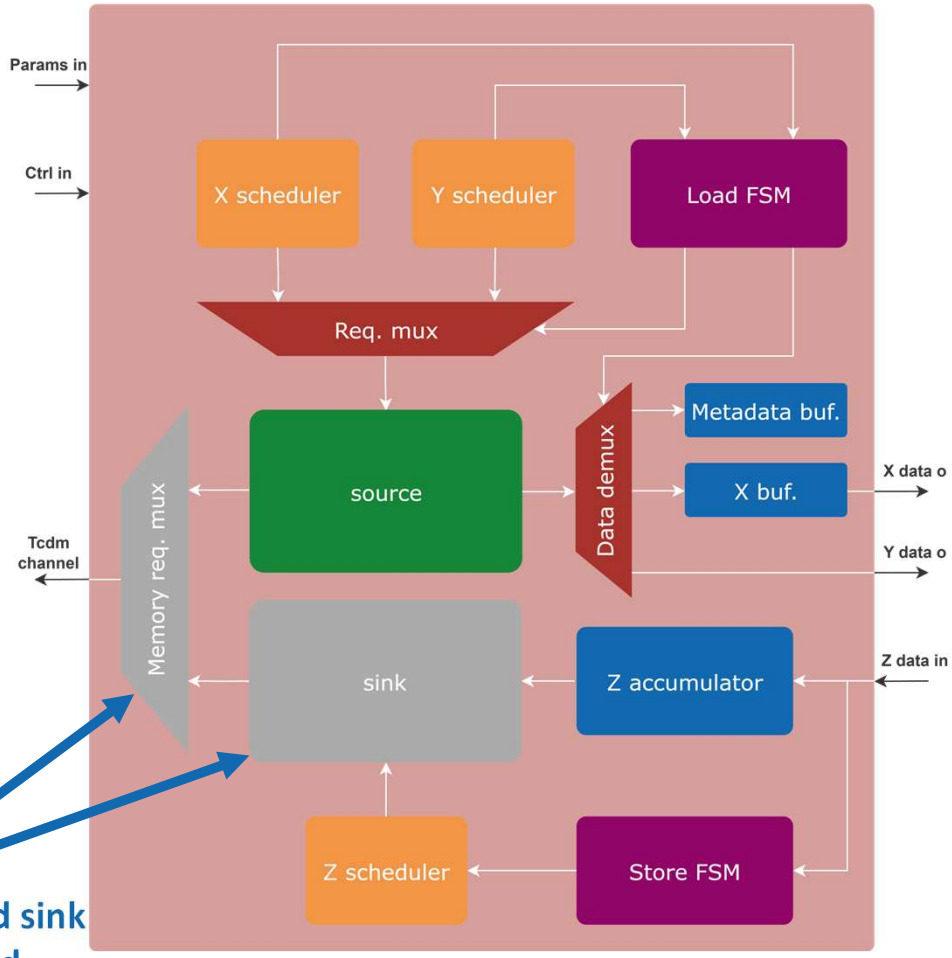
Accelerator's streamer architecture

Accelerator's streamer architecture VS datamover one



Datamover's streamer architecture

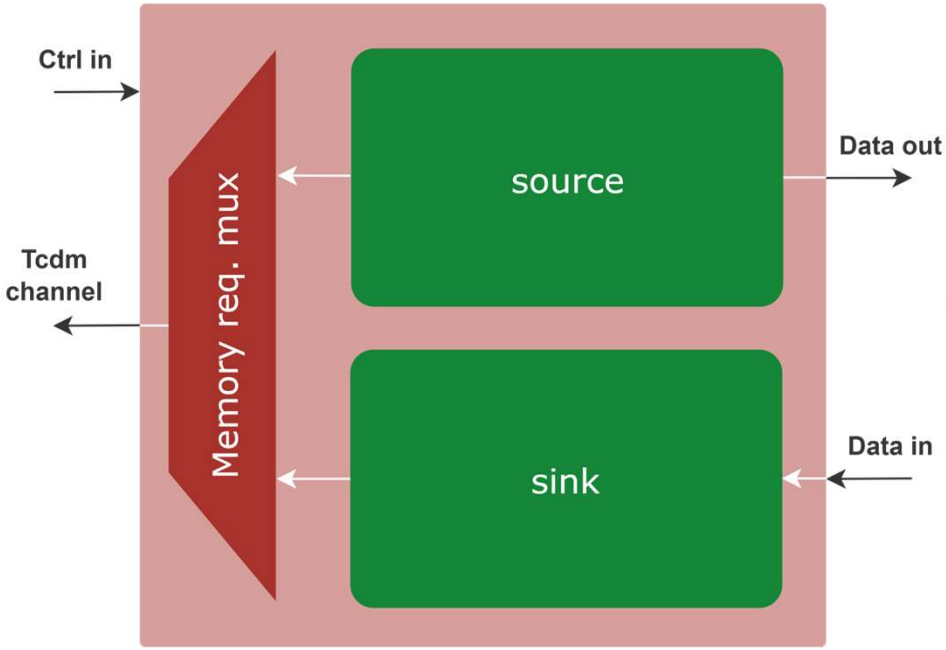
VS



Accelerator's streamer architecture

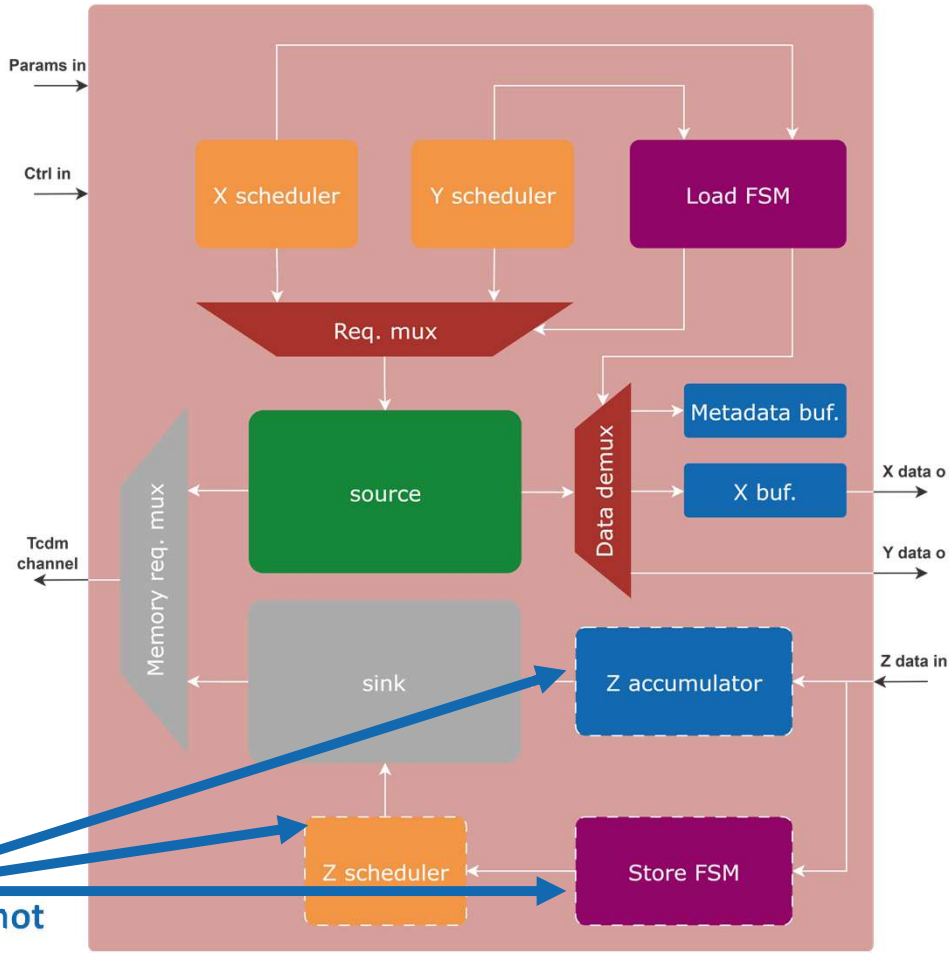
The memory mux and sink were not modified

Accelerator's streamer architecture VS datamover one



Datamover's streamer architecture

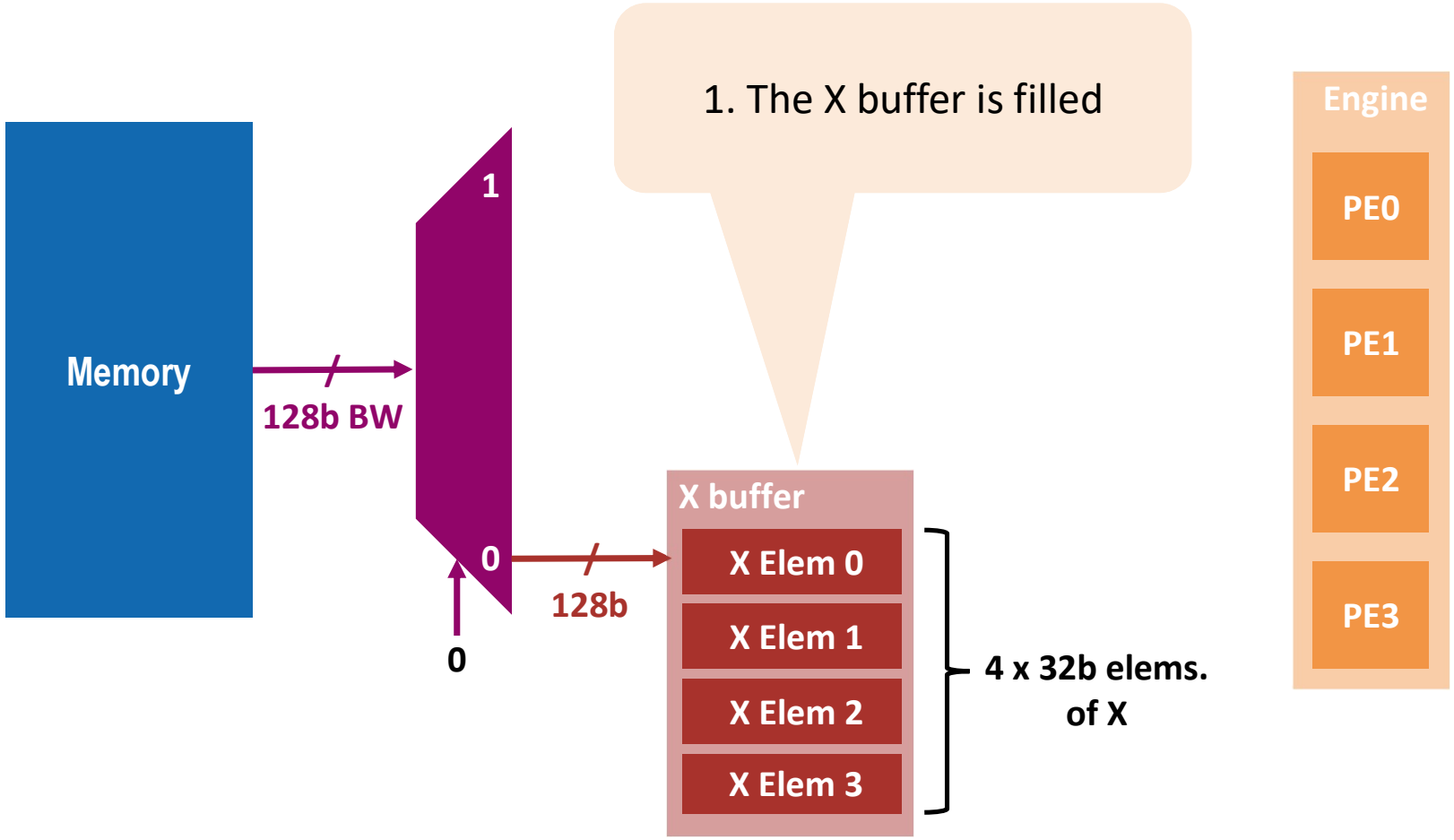
VS



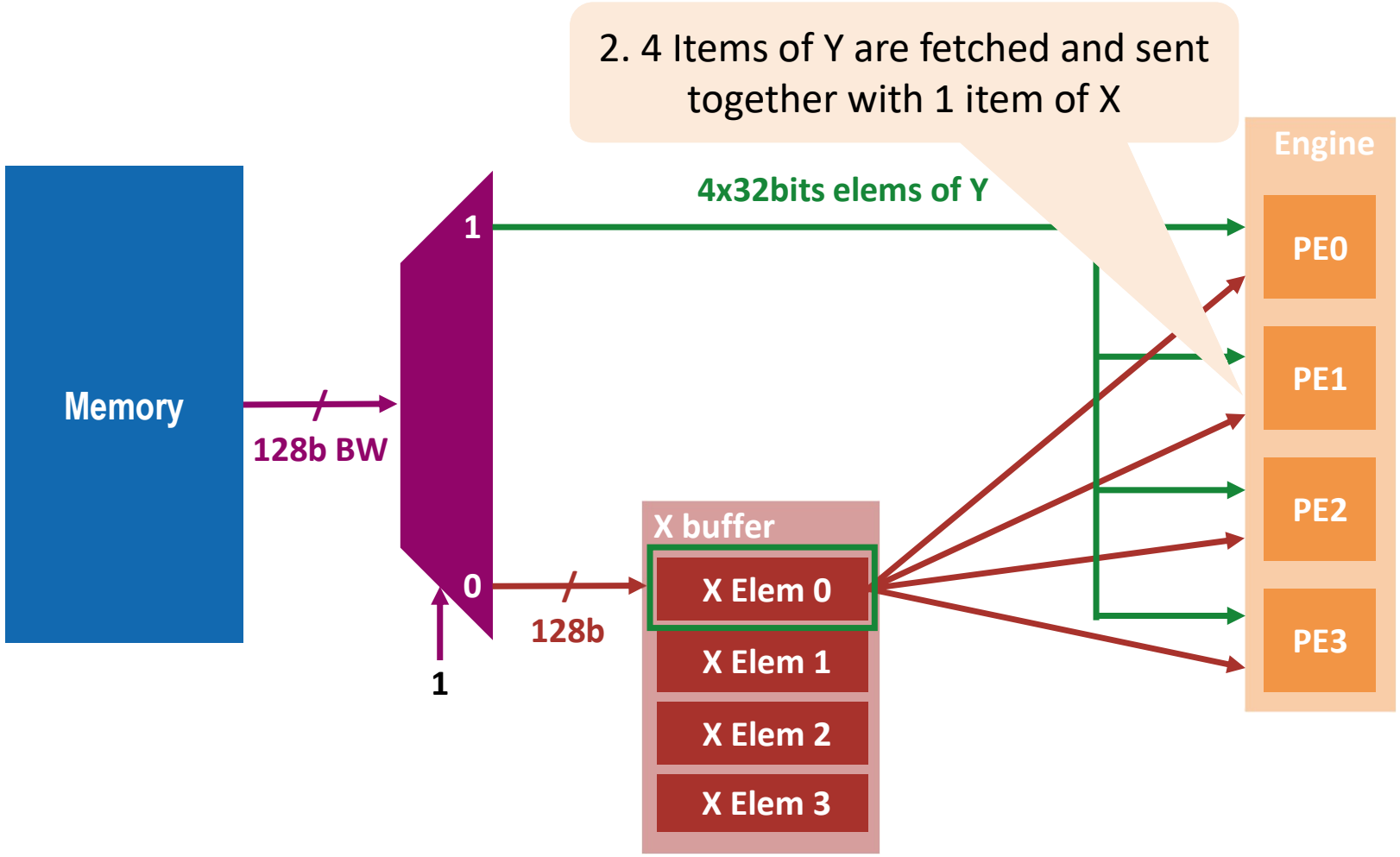
The storing part is not sparsity aware

Accelerator's streamer architecture

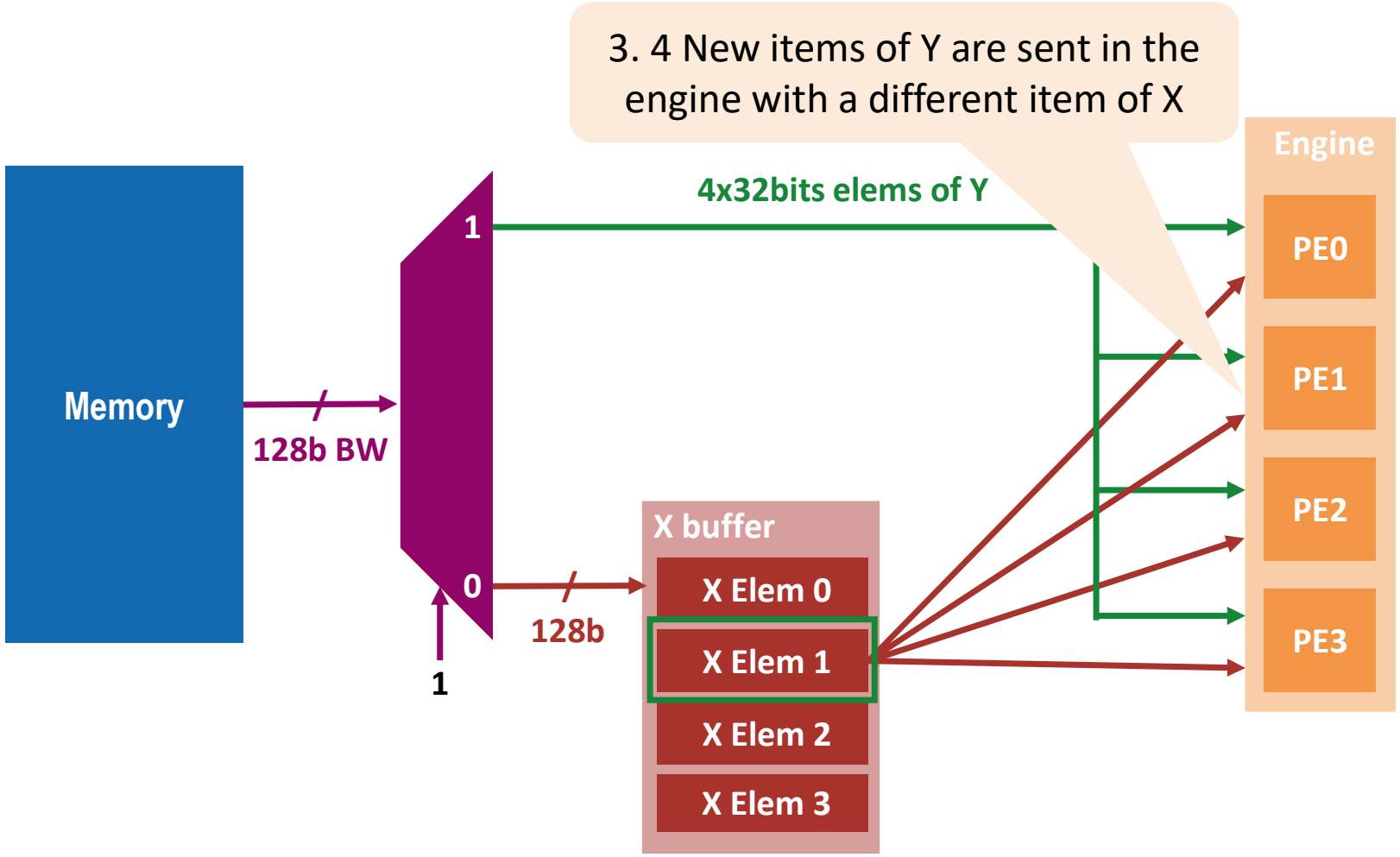
Streamer behavior –Loading part



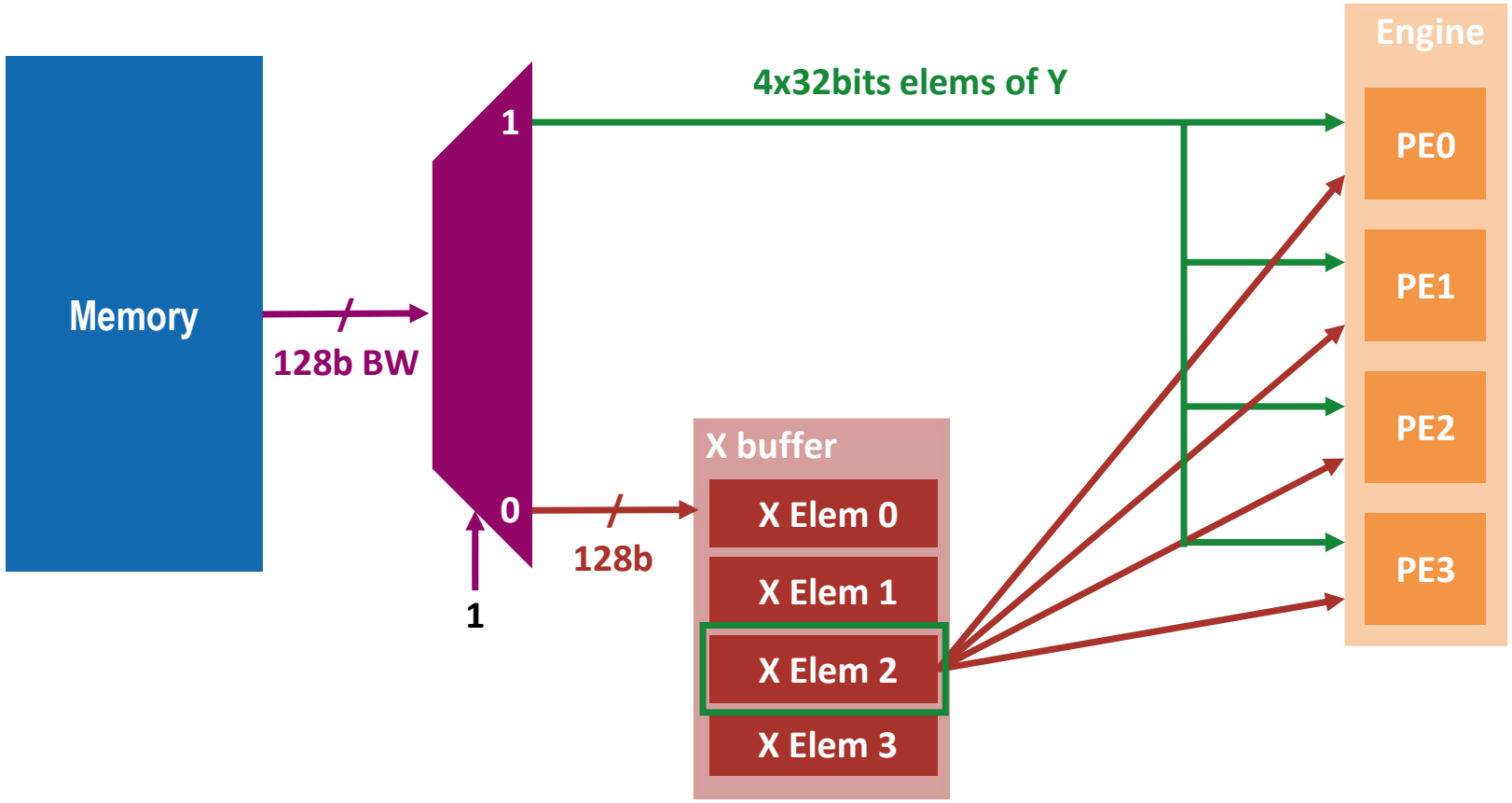
Streamer behavior –Loading part



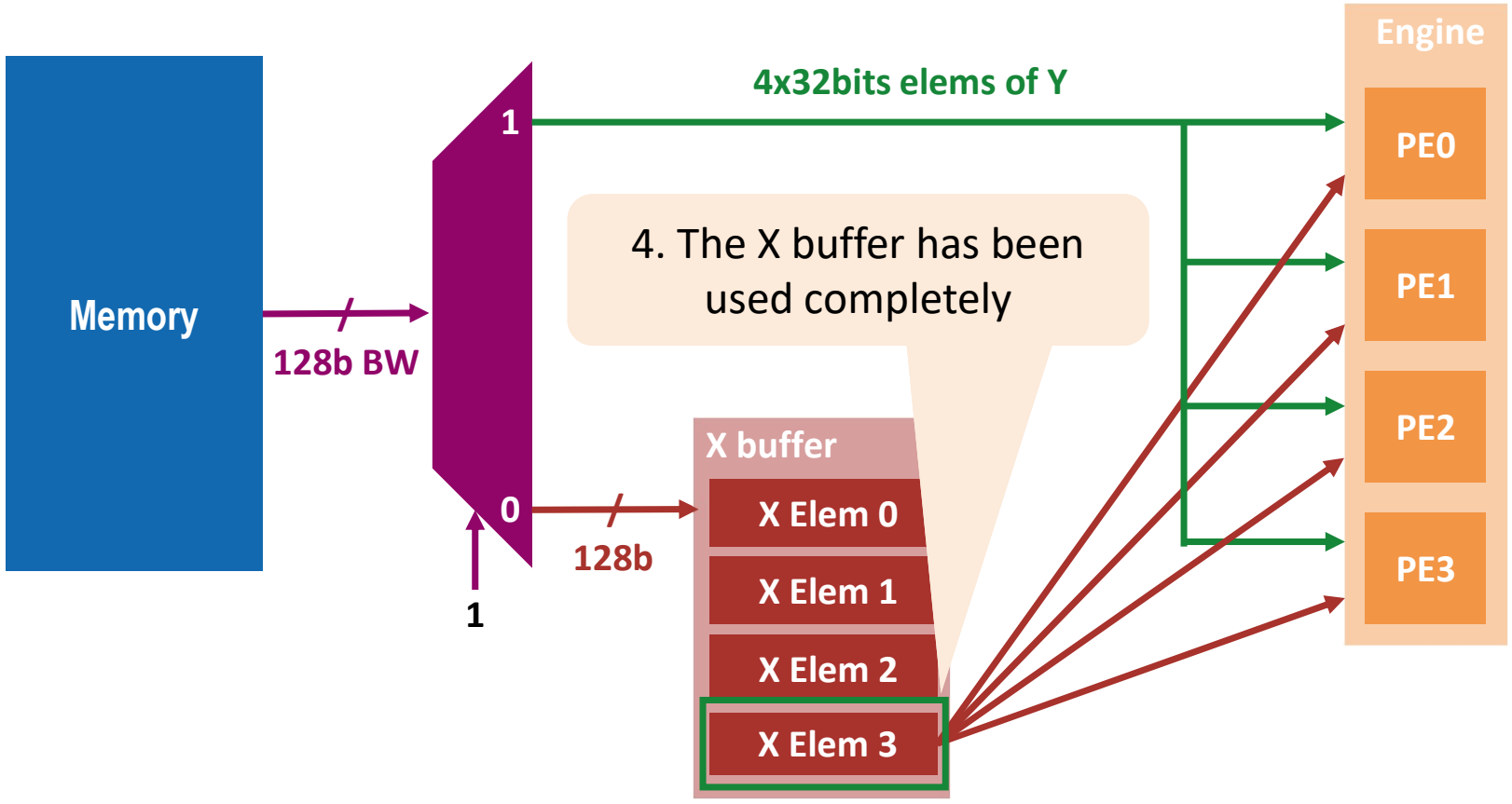
Streamer behavior –Loading part



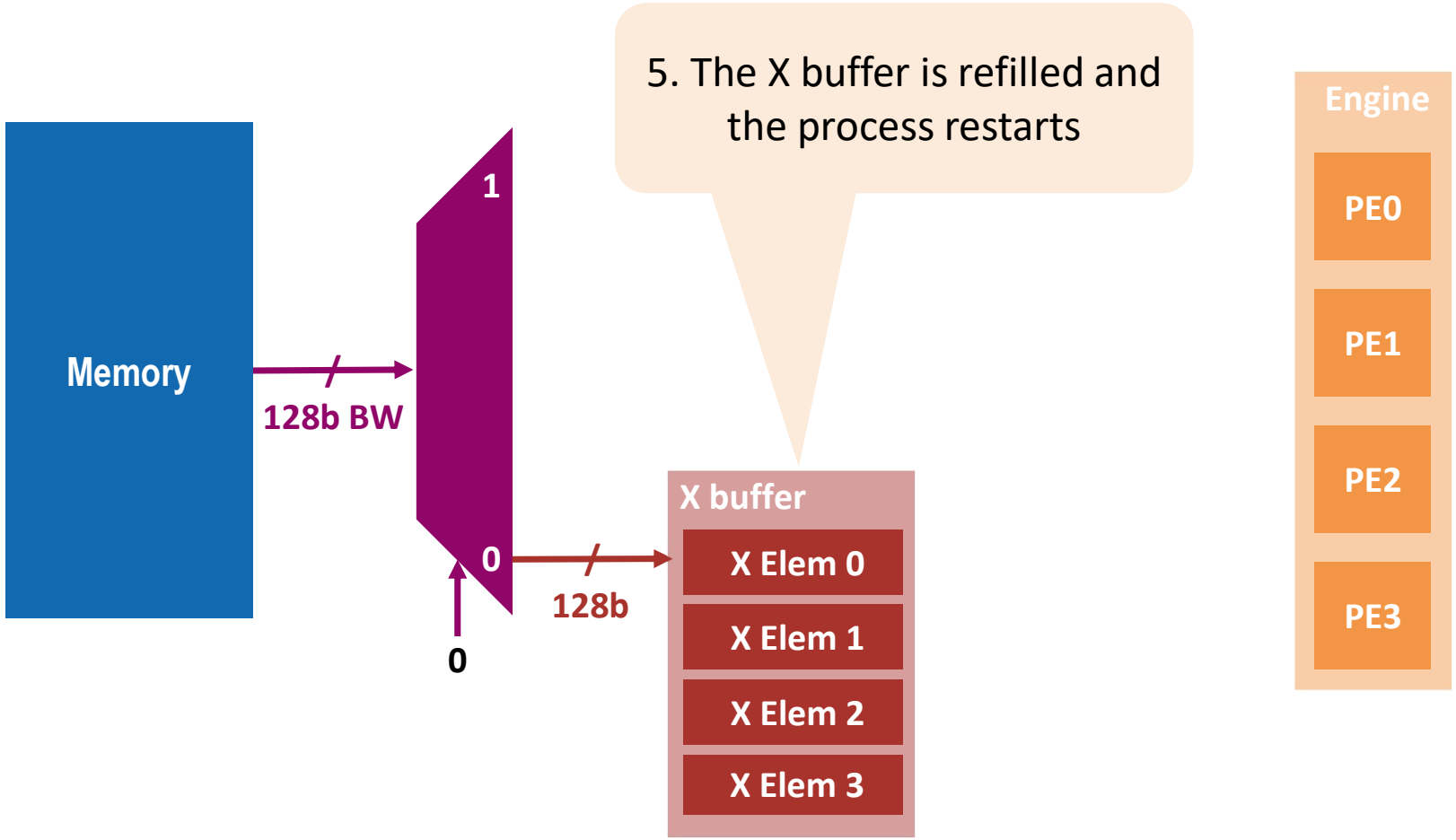
Streamer behavior –Loading part



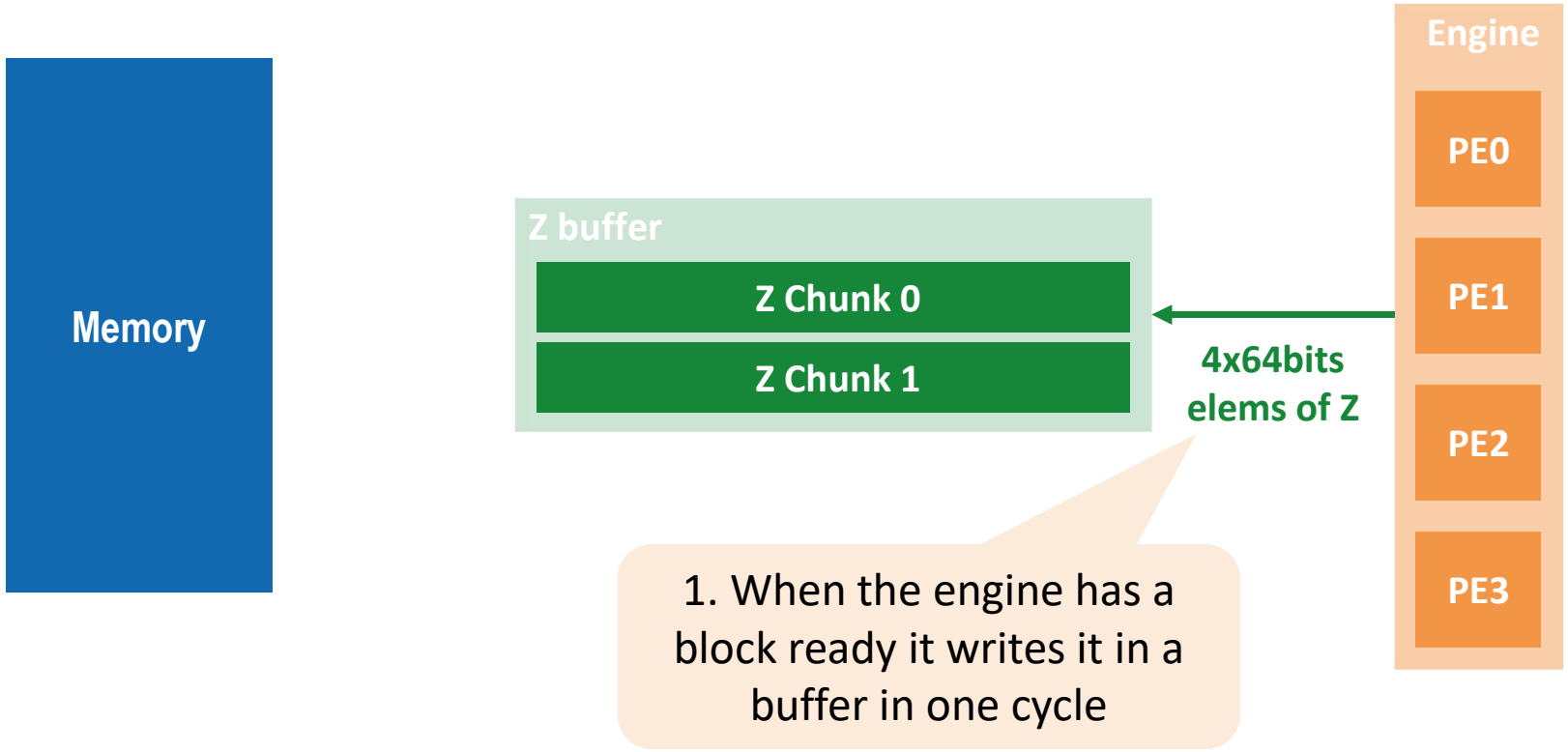
Streamer behavior –Loading part



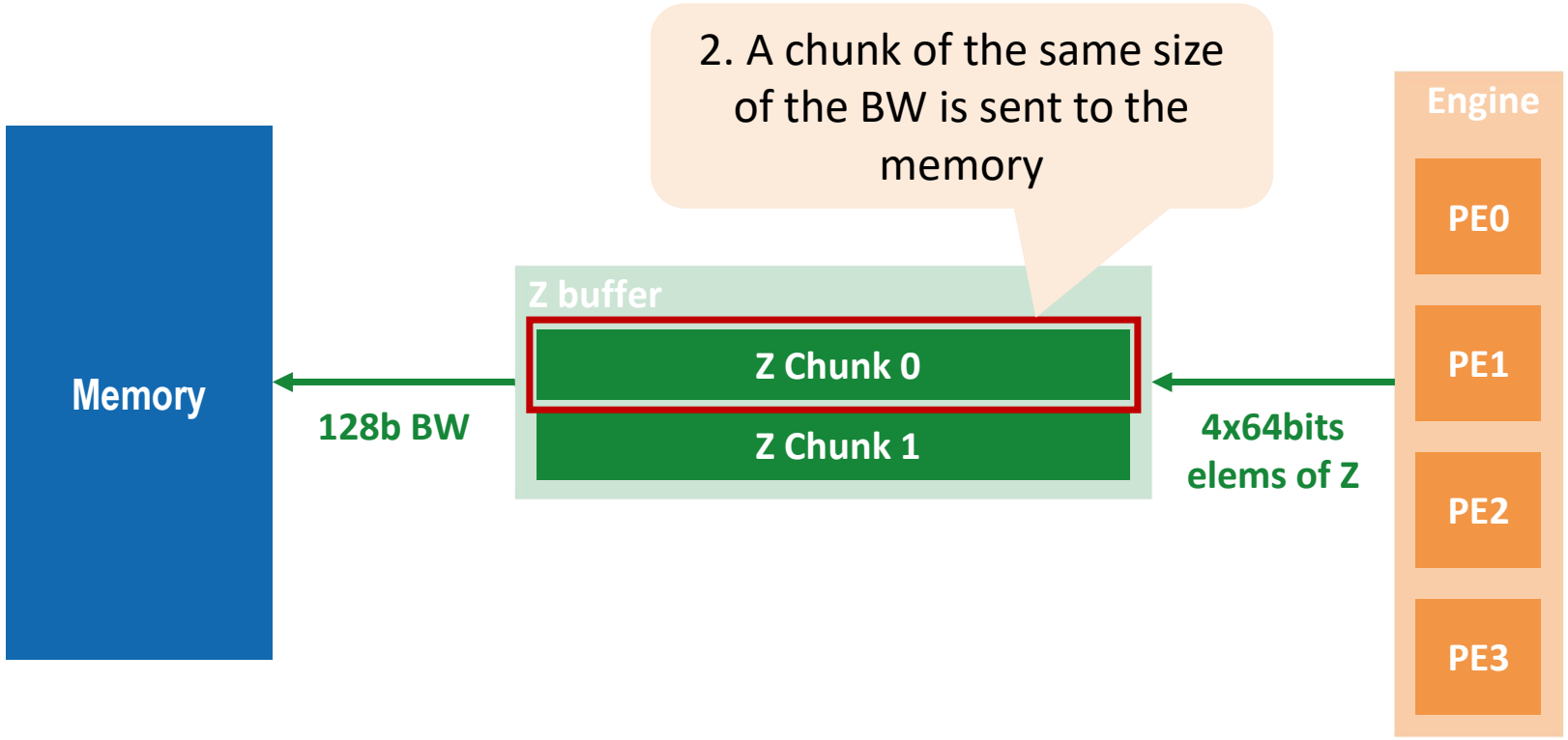
Streamer behavior –Loading part



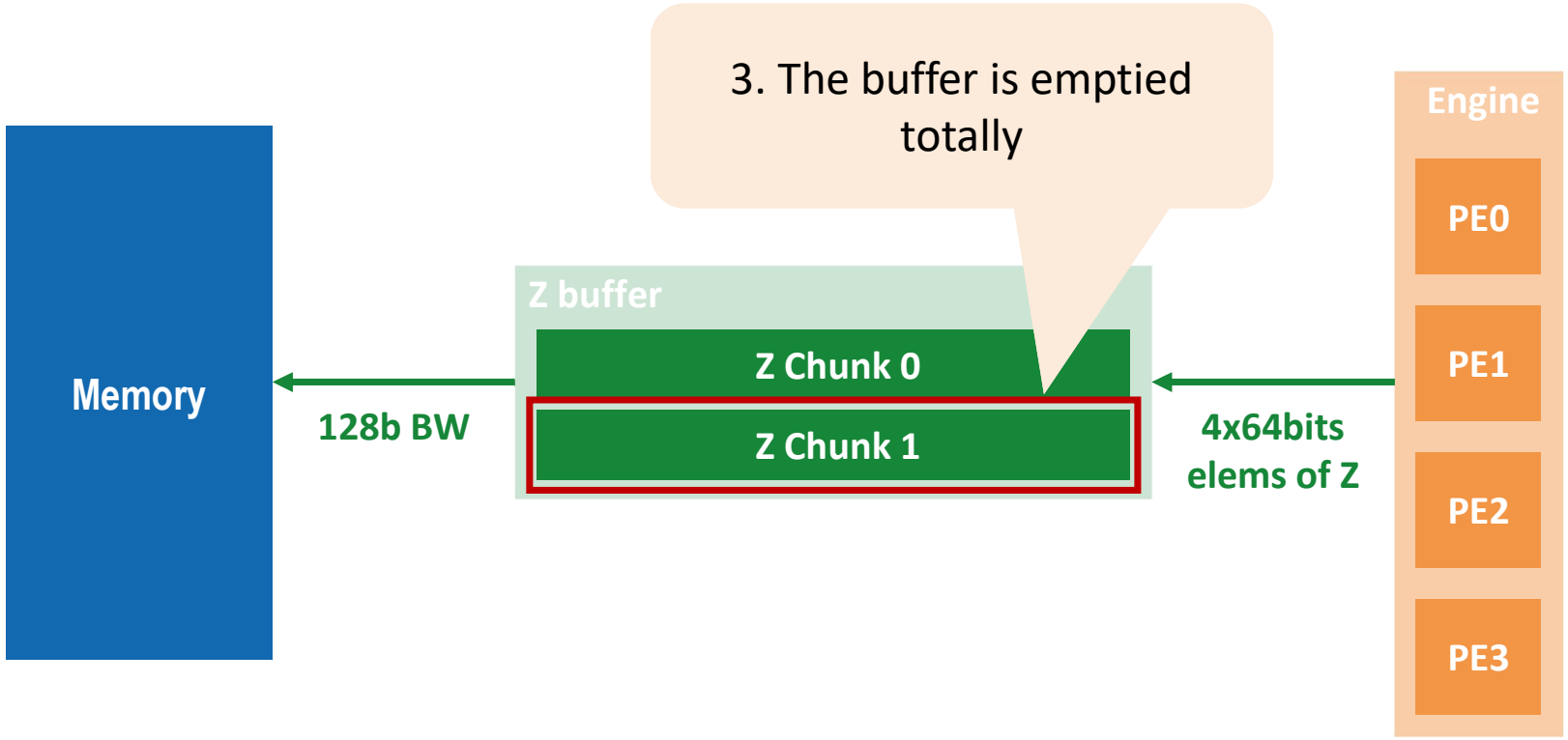
Streamer behavior –Storing part



Streamer behavior –Storing part

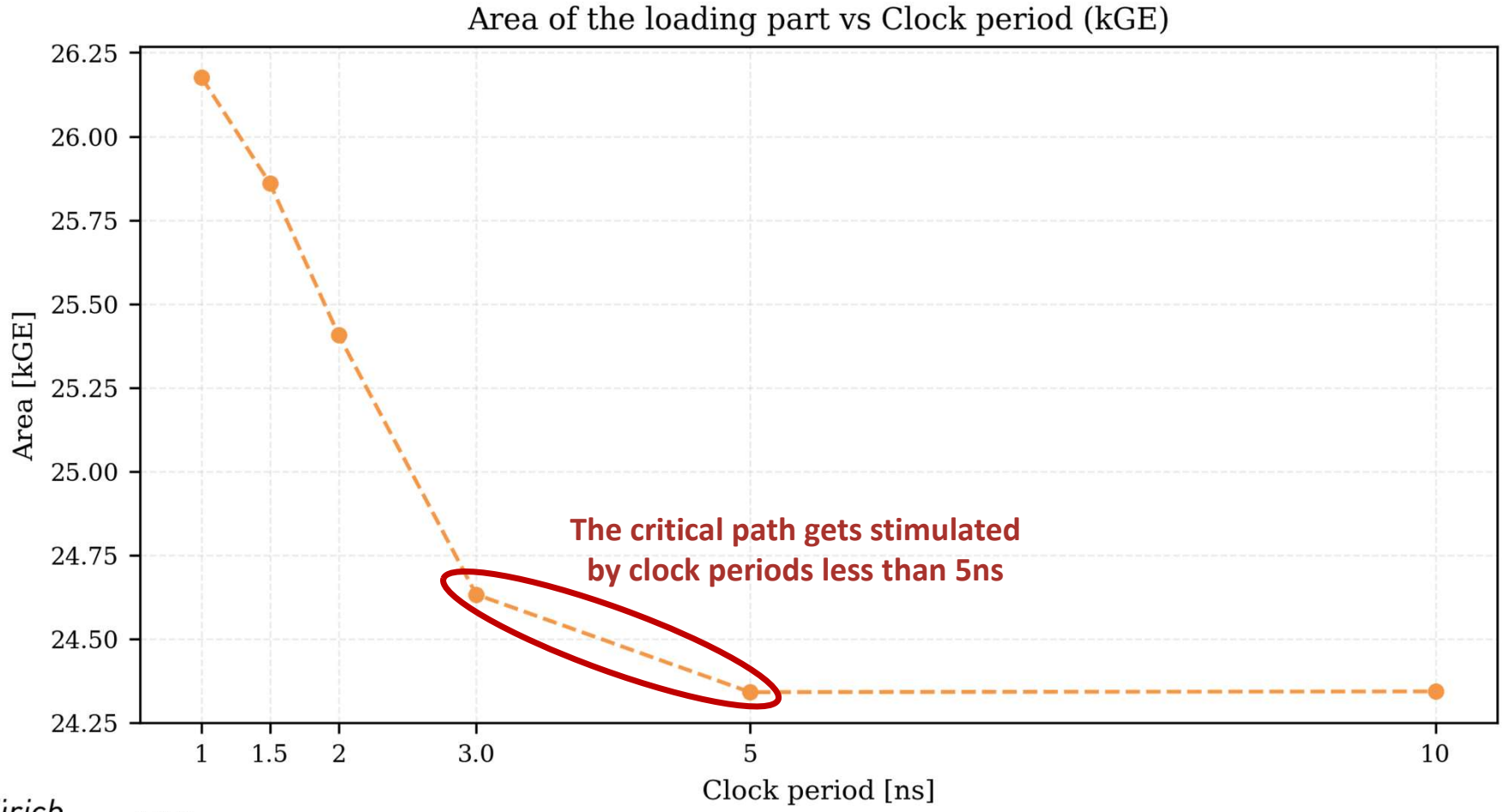


Streamer behavior –Storing part



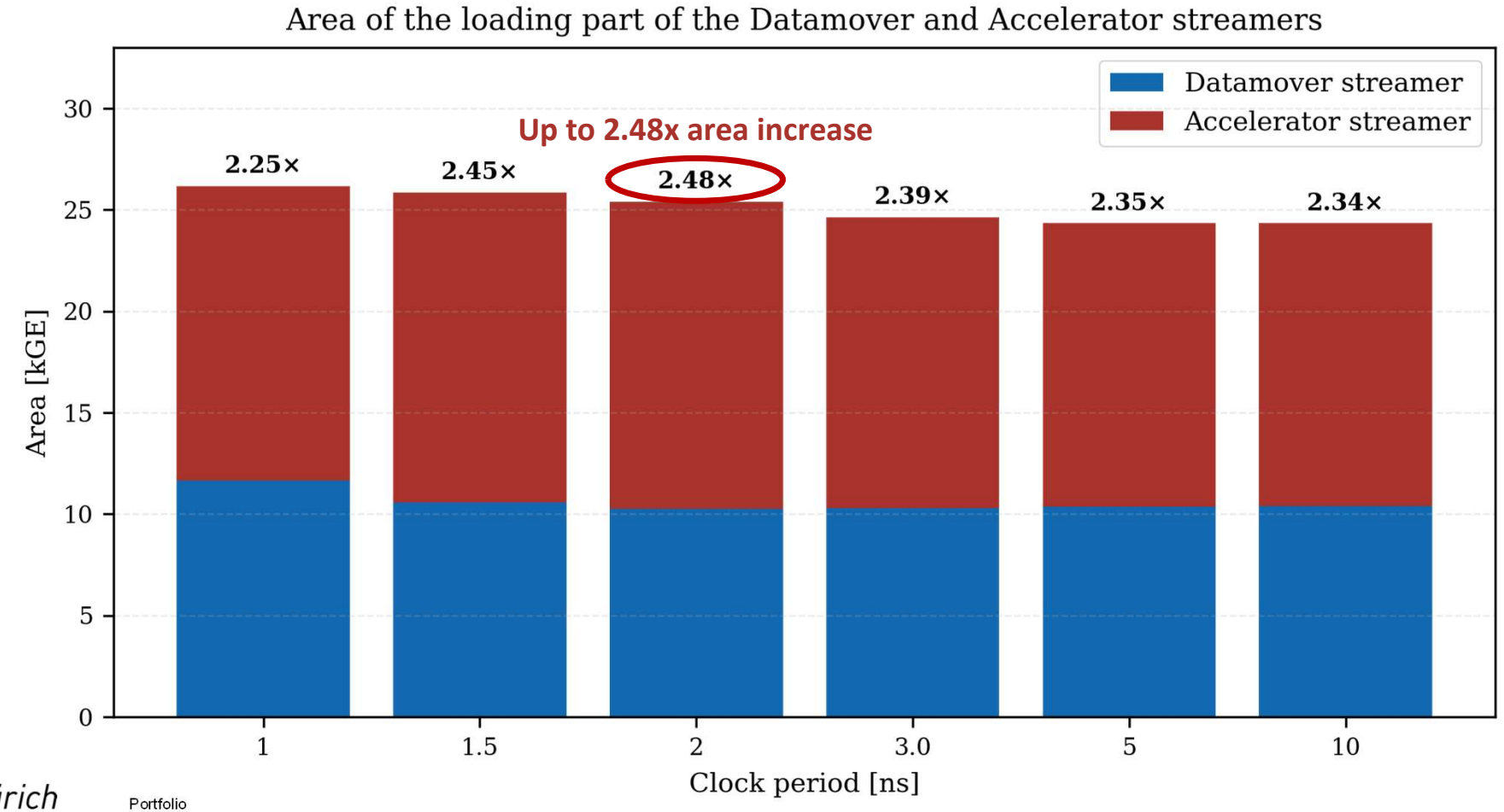
Synthesis results- AT plot

The plot reports the area considering only the loading part which was made sparsity-aware; synthesis was performed using the slow corner of the cells.



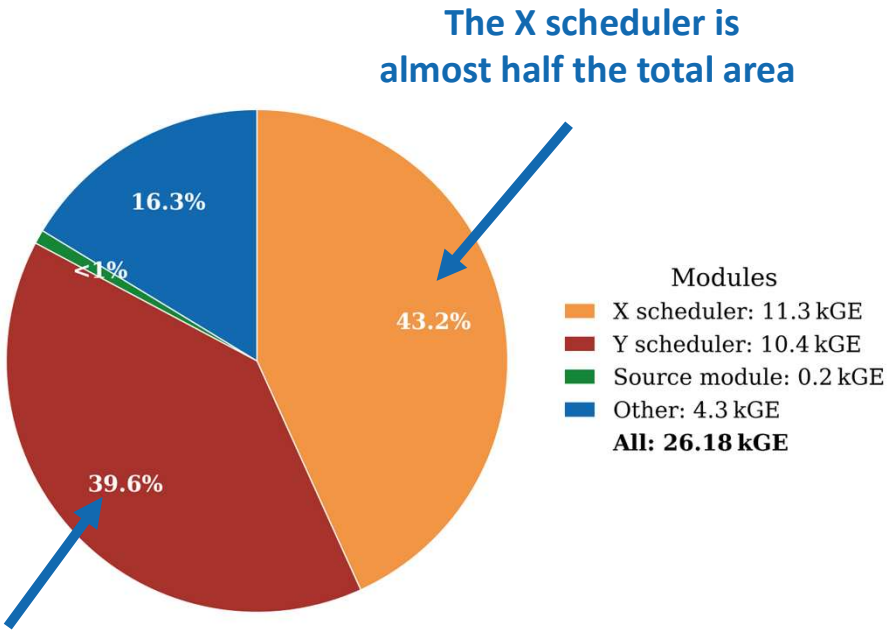
Synthesis results- Streamer VS Datamover

The plot reports the area difference only considering the loading part of the two designs, which was made sparsity-aware.



Synthesis results- Area contributions

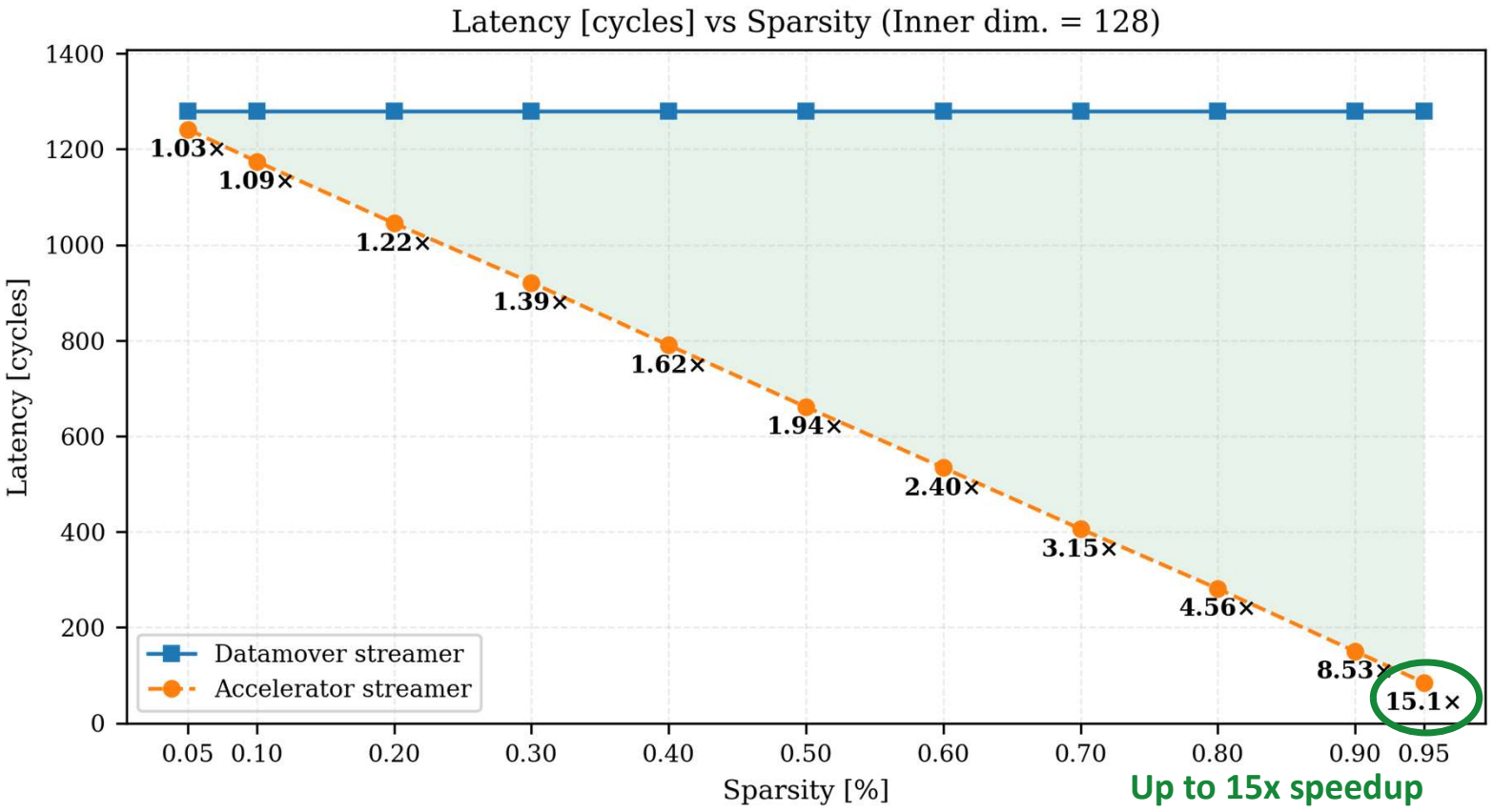
- **Adding intelligence is expensive**
  - The biggest area contributions come from the schedulers
- **Different function of the source module**
  - The source was reduced of about 10 times because it's not "smart" anymore.



Area of the loading part of the streamer at a frequency of 1GHz

RTL results –Latency VS sparsity

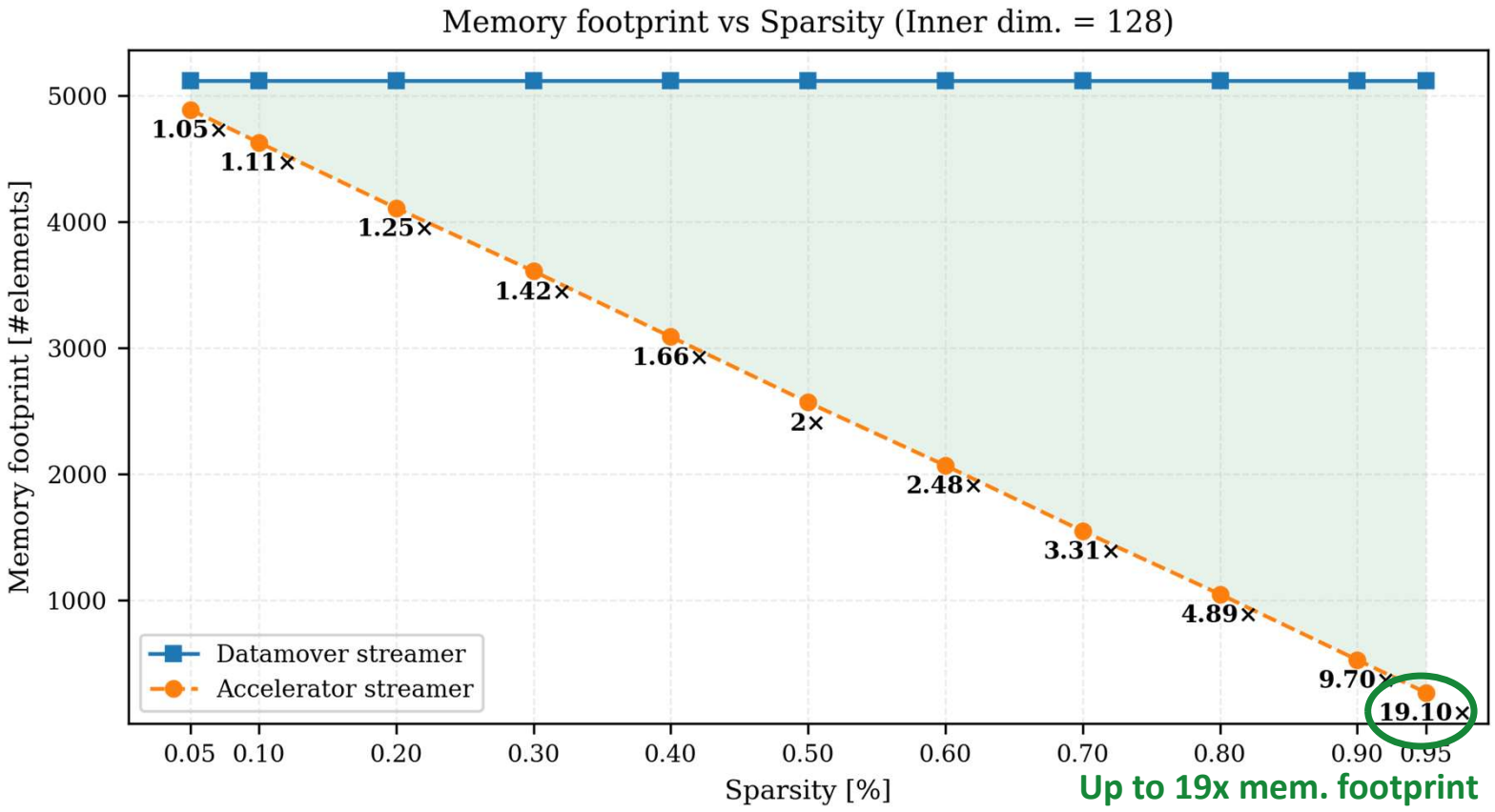
- **Assumptions:**
  - MAC units = 4
  - X rows = 2
  - Y columns = 16
  - Inner dim. = 128
  - Metadata chunk = 128b
  - Input data size = 32b
  - Output data size = 64b
  - BW = 128b



Up to 15x speedup with 95% sparsity

RTL results –Memory footprint VS sparsity

- **Assumptions:**
  - MAC units = 4
  - X rows = 2
  - Y columns = 16
  - Inner dim. = 128
  - Metadata chunk = 128b
  - Input data size = 32b
  - Output data size = 64b
  - BW = 128b



Up to 19x mem. footprint decrease with 95% sparsity

ETH zürich



Expanding an existing SoC

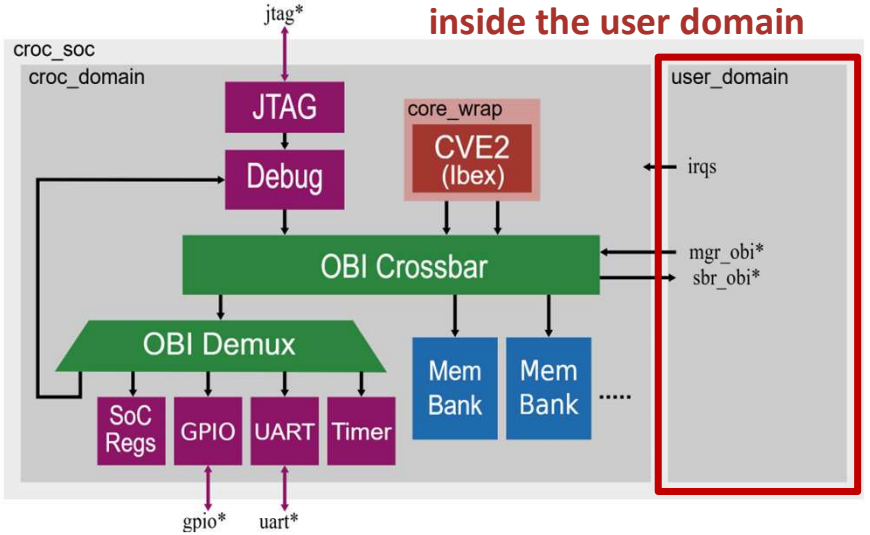
E-Bee-P-Chip -Overview

- **Context**
  - **VLSI2:** two-people project work
  - **Scope:** learn the full stack of an ASIC workflow
- **Goal**
  - Improve an existing SoC (Croc) by adding something
- **Workflow**
  - **Implement RTL:** implemented an accelerator for data compression (EBPC), increased RAM size
  - **Functional verification:** verified the functionality of the RTL with a C testbench
  - **Physical implementation:** complete backend of the ASIC
  - **Results:** comparison with the base SoC to measure speedup, area and power analysis



Logo of our chip

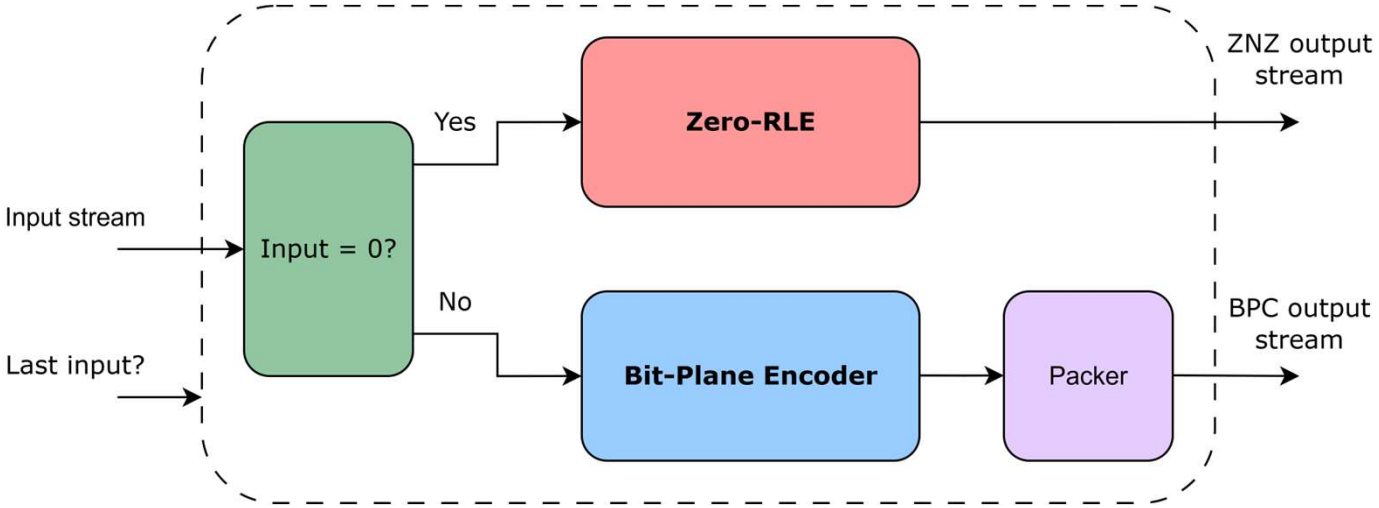
The module was implemented inside the user domain



Structure of the Croc SoC

E-Bee-P-Chip -EBPC

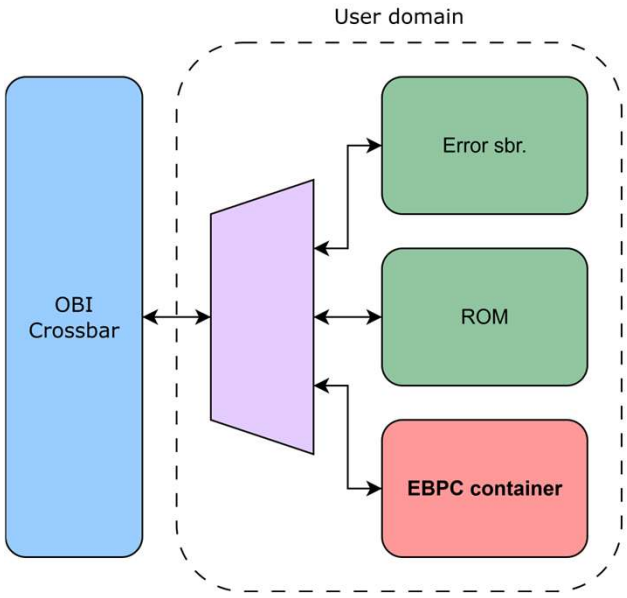
- **Ideal conditions**
  - A stream of 16-bit data items that are both sparse and correlated
- **Two modules**
  - **ZRLE:** compresses data items equal to 0
  - **BPC:** compresses nonzero-correlated data



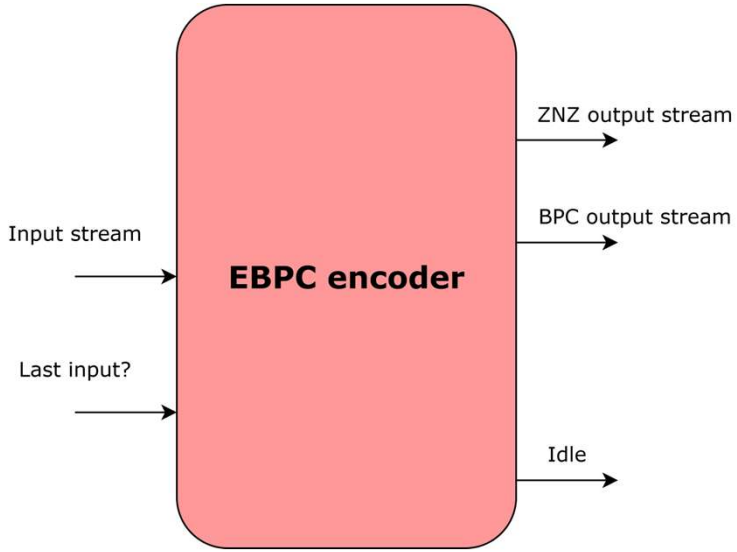
Structure of the Croc SoC

E-Bee-P-Chip –RTL implementation

- **We took a ready-to use IP from this [Github Repository]**
- **Implemented inside the user domain**
  - Inside a container that decodes the addresses of the OBI protocol
  - The container is connected to a sbr. Demux



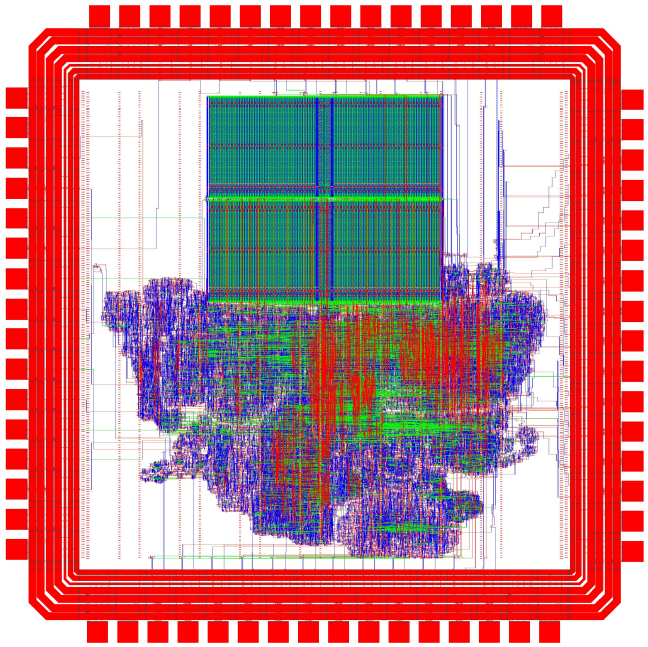
Integration of the module in the User Domain



Integration of the module in the User Domain

E-Bee-P-Chip –Backend

- **Synthesis**
  - Yosys
- **Physical backend**
  - Openroad
- **RAM banks**
  - From 2x2KB to 2x8KB
- **Clock frequency**
  - From 72MHz to 70MHz



Final layout

	Active area [mm^2]	Core area [mm^2]	Core utilization
Croc	0.78	1.76	44.4%
E-Bee-P-Chip	1.25	2.51	49.6%

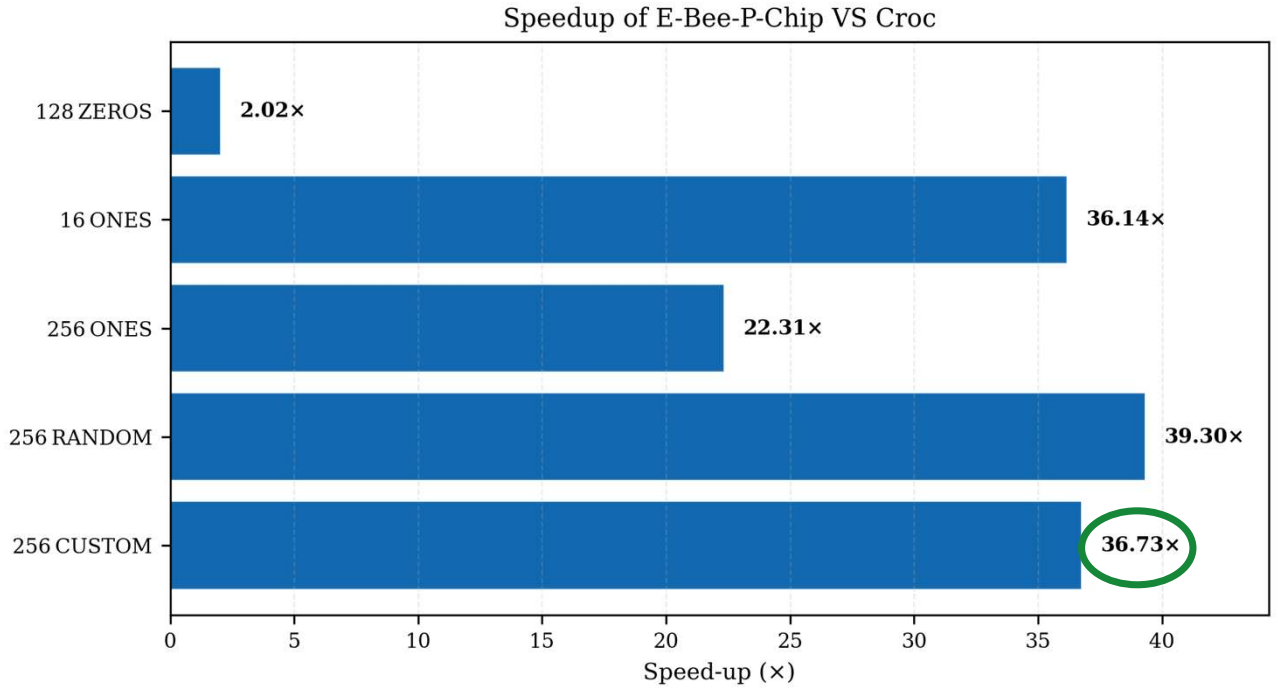
Area results

	Power [mW]
Croc	51.0
E-Bee-P-Chip	51.7

Power results

E-Bee-P-Chip –Benchmarks

- **Method**
  - C testbench that uses our accelerator VS C testbench that only uses core
- **Different use cases**
  - Custom is made of 256 16-bit sparse and correlated data items



Up to 37x speedup for the realistic case